



SEGi
University &
Colleges



College Name	University of Hertfordshire SEGi College <u>Subang Jaya</u>			
Programme Name	BACHELOR OF <u>CYBERSECURITY AND NETWORKS</u>			
Module Name	CYBER SECURITY AND NETWORKS PROJECT	Module Code	6COM2019	
		Semester	MARCH 2026	
Module Leader	MS ANIZAH ABU BAKAR	Assessment Type	PROJECT REPORT SECURE CHAT APPLICATION	
Lecturer Name	MS NUR DIANA MADINAH			
Student's declaration	I hereby certify that this assignment is my own work and where materials have been used from other resources, they have been properly acknowledged. I also understand I will face the possibility of failing the module if the content of this assignment is plagiarized.			
	No.	Name	Student ID	Signature / Initial
	1	TAN JIAJIAN	SCSJ1901781	TAN
Release Date		Submission Due Date	10/6/2026	Marks obtained:
Date Received		Student's work assessed by / date		

Module Leader’s Feedback.

Table of Contents

Chapter 1 Introduction	7
1.1 Background.....	7
1.2 Problem Statement.....	8
1.3 Research Questions and Objectives.....	9
Chapter 2 Literature Review	11
2.1 Introduction.....	11
2.2 Conceptual and Theoretical Background.....	12
2.2.1 Secure Communication Systems.....	12
2.2.2 Cybersecurity Threats in Messaging Systems.....	12
2.2.3 End-to-End Encryption (E2EE)	12
2.2.4 Advanced Encryption Standard (AES)	13
2.2.5 RSA Cryptography.....	13
2.2.6 Diffie-Hellman Key Exchange.....	14
2.3 Review of Existing Systems and Technologies	15
2.3.1 Telegram	15
2.3.2 WeChat	15
2.3.3 LINE	16
2.3.4 Proposed System: ChipChat.....	16
2.4 Comparative Analysis.....	17
Chapter 3 Methodology	18
3.1 Research / Development Approach.....	18
3.2 System Design	21
3.2.1 System Architecture.....	21
3.2.2 Flowchart	24
3.2.3 ERD.....	26
3.2.4 Use Case Diagram.....	28
Chapter 4 Implementation	30
4.1 System Architecture Overview	30
4.2 User Interface Implementation.....	31
4.3 Admin Interface Implementation.....	36
4.4 Security Feature Implementation.....	38
4.5 Network Security Verification.....	43
Chapter 5 Discussion	44
5.1 Discussion of Objective 1	44
5.2 Discussion of Objective 2	45
5.3 Discussion of Objective 3	47
5.4 Strengths of the Proposed System.....	49
5.5 Limitation of Proposed System.....	50

Chapter 6 Conclusion	51
6.1 Summary of the Study / Project	51
6.2 Achievement of Objectives	52
6.3 Recommendations for Future Work.....	53
6.4 Final Remarks	54
References	55
Gantt Chart	57
Appendix	58

Table of Figure

Figure 1 Agile Development Methodology	18
Figure 2 ChipChat System Architecture	21
Figure 3 ChipChat System Flowchart.....	24
Figure 4 Entity Relationship Diagram	26
Figure 5 Use Case Diagram	28
Figure 6 Login Page.....	31
Figure 7 Registration Page.....	31
Figure 8 Password Recovery Page.....	32
Figure 9 Main Chat Page	32
Figure 10 Main Chat Interface.....	33
Figure 11 Media Sharing Sender Side	33
Figure 12 Media Reception Receiver Side	34
Figure 13 Edit Profile Page.....	34
Figure 14 Contact Information Page.....	35
Figure 15 Admin Login Page.....	36
Figure 16 Admin Registration Page.....	36
Figure 17 Security Operations Center Dashboard	37
Figure 18 Database Security Evidence	37
Figure 19 E2EE Monitoring Panel.....	38
Figure 20 SQL Injection Detection.....	38
Figure 21 SQL Injection Attack Log	39
Figure 22 XSS Detection	39
Figure 23 XSS Attack Log.....	40
Figure 24 Brute Force Attack Log	40
Figure 25 Replay Attack Message Capture.....	41
Figure 26 Replay Attack Simulation.....	41
Figure 27 Replay Attack Detection Log	42
Figure 28 Wireshark TLS Capture.....	43
Figure 29 E2EE Message Relay Evidence.....	44
Figure 30 Discussion of SQL Injection Attack Log	45
Figure 31 Discussion of XSS Attack Log.....	45
Figure 32 Discussion of Brute Force Attack Log	46
Figure 33 Discussion of Replay Attack Detection Log	46
Figure 34 Encryption and Delivery Time Testing	47
Figure 35 Gantt Chart	57

List of Table

Table 1 Research Objectives.....	10
Table 2 Security Feature Comparison.....	17
Table 3 Functional Feature Comparison.....	17
Table 4 Development Environment.....	30
Table 5 Performance Testing Results	48

Chapter 1 Introduction

1.1 Background

In today's world, online communication has become an important part of daily life. Messaging, file sharing and connecting with people across the world can now happen instantly through social media, emails and chat applications. These technologies have changed the way communication works in both personal and professional settings. However, while online communication has become faster and more convenient, it has also created serious security risks. Cyberattacks, data leaks and hacking are becoming more common and without proper security, important information such as passwords, private messages and business data can easily be exposed.

To solve these issues, secure chat applications have become more important than ever. The main purpose of these applications is to ensure that messages can only be read by the sender and receiver. This is possible through end-to-end encryption (E2EE), which protects messages during transmission. Even if a message is intercepted, it cannot be understood without the correct decryption key. This helps protect users from hackers, man-in-the-middle attacks and unauthorized access.

This project develops a secure chat system by combining trusted encryption methods including RSA-OAEP and AES-GCM. RSA-OAEP is used for secure key exchange, while AES-GCM provides fast and secure message encryption. By combining these methods, the system creates stronger protection against data theft, message tampering and unauthorized access.

The main goal of this project is to create a chat system that is both useful and secure. By applying cryptography together with web development and network security concepts, this project demonstrates how safe and private communication can be achieved in today's digital environment.

1.2 Problem Statement

Nowadays, it's almost impossible to imagine life without online communication. Billions of messages are sent every day through chat apps, emails and social platforms. This shift has made it incredibly easy to connect with people, both personally and professionally. However, that same convenience also brings new dangers. Cyberattacks like phishing, eavesdropping and man-in-the-middle (MITM) attacks are happening more often than ever. Recent cybersecurity studies report that more than 80% of data breaches involve unauthorized access to private or organizational communication (Ali and Sagheer, A. (2017). Clearly, there's a growing need for communication systems that are both private and secure.

Many popular chat services still rely on centralized servers to store user messages. While some of them advertise that they use encryption, not all actually implement full end-to-end encryption (E2EE). That means messages could still be accessed by the company itself or by hackers if the servers are compromised (Bais, A.S., Sao, A., Jain, H., Sahu, S. and Bajpai, P. (2024). For instance, if encryption keys are stored on the same server, a single security breach could reveal every user's conversation. Weak key management and incomplete encryption can easily lead to privacy leaks, identity theft, or even financial loss.

This isn't just a personal issue either. Businesses, schools and government offices also exchange sensitive data online every day. Without a secure messaging system, they face real risks from espionage to data corruption or even surveillance. That's why developing a communication platform that is secure, efficient and easy to use has become so important in our connected world.

To tackle these challenges, this project focuses on creating a Secure Chat Application that uses multiple layers of encryption such as Hybrid encryption using RSA-OAEP and AES-GCM. Each algorithm plays a different but essential role: AES-GCM provides fast data encryption, RSA-OAEP secures key exchange. Research by Perkasa, M.J.P., Ramdan, H.M., Maliki, A.J. and Somantri. (2025), Carranza, H., Bustamante, M., Carranza, A. and Muniganti, S., (2024). and Yusfrizal and Meizar (2024) confirm that combining these algorithms can offer reliable protection in modern messaging systems. With its focus on end-to-end encryption and secure key exchange, this project aims to show how strong cryptography can make online communication both safer and more trustworthy.

1.3 Research Questions and Objectives

With growing public concern about online privacy and data security, the demand for secure and encrypted communication systems continues to increase. This project aims to develop a secure chat application that protects confidentiality, integrity and authenticity of data through encryption and secure key exchange methods. To support this research, several research questions have been proposed.

How can end-to-end encryption be effectively implemented in chat applications to ensure that only authorized users can access transmitted messages?	This question focuses on how to correctly apply end-to-end encryption (E2EE) to ensure that only the sender and receiver can read messages. It examines secure key generation, key exchange and how encryption and decryption can be performed on the user's device rather than the server.
In real-time communication, which combination of encryption algorithms such as RSA-OAEP and AES-GCM achieves the best balance between performance and security?	This question aims to explore which encryption combination provides the best speed and security for live chat. AES-GCM encryption is fast and suitable for message encryption, while RSA-OAEP is used for key exchange. The goal is to find the most efficient and secure hybrid encryption method.
How can the system prevent unauthorized access and common cyberattacks such as man-in-the-middle attacks, replay attacks, SQL injection and brute-force attacks during communication?	The focus is on TLS secure communication, RSA-OAEP key exchange, AES-GCM encryption, hashing and identity verification mechanisms to prevent man-in-the-middle attacks and message interception.
How can usability and efficiency be balanced while maintaining high encryption standards?	This question explores how to ensure strong encryption while maintaining system usability. It focuses on reducing latency, simplifying the interface and ensuring encryption functions automatically without compromising the usability of the chat

	application.
--	--------------

Table 1 Research Objectives

Objectives

1. To design and develop a secure chat application using end-to-end encryption with RSA-OAEP and AES-GCM cryptographic algorithms.
2. To ensure message confidentiality, integrity and authenticity through secure key exchange and encryption technologies.
3. To evaluate the application's performance and user experience in terms of encryption speed, message delivery and ease of use.

Chapter 2 Literature Review

2.1 Introduction

This chapter reviews existing literature related to secure communication systems, end-to-end encryption (E2EE), cryptographic algorithms, cybersecurity threats and secure messaging technologies. This review helps explain the security concepts used in this project and shows how existing chat systems protect user communication for the development of the proposed secure chat application.

As more communication activities move online, protecting sensitive information has become increasingly important for both individuals and organizations. Online messaging applications are widely used for personal communication, business collaboration and information sharing. However, these applications are frequently targeted by cyberattacks such as eavesdropping, man-in-the-middle attacks, replay attacks and unauthorized access. As a result, researchers have increasingly focused on developing secure communication systems that provide confidentiality, integrity and authentication while maintaining usability and performance.

This chapter discusses the fundamental concepts of secure communication, cryptography, end-to-end encryption and cybersecurity threats. It also reviews previous studies related to secure chat applications and compares existing technologies used in modern messaging platforms. Lastly, the chapter compares existing messaging technologies and discusses the security concepts used in the proposed ChipChat system.

2.2 Conceptual and Theoretical Background

2.2.1 Secure Communication Systems

Secure communication refers to the process of transmitting information between parties while preventing unauthorized access, modification or disclosure of the transmitted data. The primary objectives of secure communication are confidentiality, integrity, authentication and non-repudiation. These objectives ensure that only authorized users can access information, transmitted data remains unchanged during communication and users can verify the identity of communication participants.

Modern communication systems rely heavily on cryptographic techniques to achieve these security objectives. According to Deepika et al. (2021), cryptography plays a fundamental role in protecting sensitive information by transforming readable data into encrypted ciphertext that can only be accessed using the appropriate decryption key. The increasing number of cyberattacks targeting communication systems has made cryptography an essential component of secure messaging applications.

2.2.2 Cybersecurity Threats in Messaging Systems

Messaging applications face various cybersecurity threats that may compromise user privacy and system security. According to Trend Micro (2025) and Fortinet (2025), common cyberattacks targeting communication systems include man-in-the-middle attacks, phishing attacks, replay attacks, brute-force attacks, malware distribution and social engineering attacks.

Among these threats, man-in-the-middle attacks are particularly dangerous because attackers attempt to intercept communication between two parties without their knowledge. Replay attacks involve capturing valid communication data and retransmitting it to deceive the system, while brute-force attacks attempt to gain unauthorized access through repeated password guessing. These threats highlight the importance of implementing strong encryption, secure authentication and attack detection mechanisms in modern communication platforms.

2.2.3 End-to-End Encryption (E2EE)

End-to-end encryption (E2EE) is a communication security mechanism that ensures messages are encrypted on the sender's device and decrypted only on the recipient's device. During transmission, neither the communication server nor any third party can access the plaintext content of the messages. Pancoro (2024) states that E2EE provides one of the highest levels

of privacy protection in modern messaging systems because the encryption keys remain under the control of communicating users.

Similarly, Carranza et al. (2024) emphasize that E2EE significantly reduces the risk of unauthorized access by preventing service providers, attackers and intermediaries from viewing message contents. Harshvardhan et al. (2023) further highlighted that secure messaging applications implementing end-to-end encryption can protect communication privacy by ensuring that only intended users are able to access message contents. Even if communication data is intercepted during transmission, attackers are unable to understand the information without the corresponding decryption keys. As a result, many modern messaging platforms such as WhatsApp, Signal and Telegram have adopted E2EE to improve user privacy and communication security.

2.2.4 Advanced Encryption Standard (AES)

The Advanced Encryption Standard (AES) is a symmetric encryption algorithm widely used for protecting digital information. AES encrypts and decrypts data using the same secret key and is recognized for its high performance, efficiency and security. According to Perkasa et al. (2025), AES-256 provides strong protection against brute-force attacks while maintaining fast encryption and decryption speeds, making it suitable for real-time communication systems.

Yusfrizal et al. (2018) further explain that AES is commonly used for message encryption because of its computational efficiency compared to asymmetric cryptographic algorithms. In secure messaging systems, AES is often combined with other cryptographic mechanisms to create hybrid encryption architectures that provide both security and performance advantages.

2.2.5 RSA Cryptography

RSA is an asymmetric cryptographic algorithm that uses a public key and a private key for encryption and decryption. Unlike symmetric encryption algorithms, RSA allows secure communication between users without requiring them to share a secret key beforehand. Deepika et al. (2021) explain that RSA is particularly effective for secure key exchange and digital authentication because the public key can be distributed openly while the private key remains confidential.

In secure messaging applications, RSA is commonly used to exchange encryption keys

securely before message transmission. This allows users to establish protected communication channels while minimizing the risk of key compromise. Due to its security and reliability, RSA remains one of the most widely implemented public-key cryptographic algorithms in communication systems.

2.2.6 Diffie-Hellman Key Exchange

Diffie-Hellman is a cryptographic key exchange protocol that enables two parties to establish a shared secret over an insecure communication channel. Unlike RSA, Diffie-Hellman focuses on generating a common secret key rather than directly encrypting messages. According to Yusfrizal et al. (2018), combining Diffie-Hellman with AES encryption provides a secure and efficient approach for protecting communication data.

Perkasa et al. (2025) demonstrated that Diffie-Hellman key exchange can enhance the security of microservice-based chat applications by securely generating encryption keys before message transmission. Although different secure communication systems may adopt different key exchange mechanisms, Diffie-Hellman remains one of the most influential protocols in the field of cryptography and secure communication. Mokhtarnameh et al. (2010) compared several key exchange and authentication protocols and concluded that secure key exchange mechanisms play a critical role in protecting communication channels against unauthorized access.

Although Diffie-Hellman is widely used in secure communication systems, the proposed ChipChat system utilizes RSA-OAEP as the primary key exchange mechanism.

2.3 Review of Existing Systems and Technologies

Different messaging applications are widely used today to support online communication. Popular platforms such as Telegram, WeChat and LINE provide users with instant messaging, media sharing and real-time communication services. Although these applications offer convenience and security features, they differ in terms of encryption mechanisms, privacy protection and security management. This section reviews these existing systems and compares their security characteristics with the proposed ChipChat application.

2.3.1 Telegram

Telegram is a cloud-based messaging platform that provides messaging, voice communication, file sharing and group collaboration features. According to Puyosa (2023), Telegram offers encrypted communication and supports end-to-end encryption through its Secret Chat feature. However, end-to-end encryption is not enabled by default for regular chats, as standard conversations are stored within Telegram's cloud infrastructure to support synchronization across multiple devices.

Although this architecture improves convenience and multi-device access, it also raises privacy concerns because messages are stored on Telegram's servers. Users must manually initiate Secret Chats to obtain full end-to-end encryption protection. As a result, many conversations may remain protected only through server-side encryption rather than true end-to-end encryption.

2.3.2 WeChat

WeChat is one of the largest messaging applications in the world and provides messaging, social networking, digital payment and business communication functions. According to Puyosa (2023), WeChat emphasizes functionality and ecosystem integration rather than privacy-focused communication.

Unlike dedicated secure messaging platforms, WeChat does not publicly provide full end-to-end encryption for standard user communications. Communication data is protected during transmission; however, messages may still be processed through centralized servers. Consequently, privacy advocates have raised concerns regarding data collection, centralized storage and user information management.

Despite these concerns, WeChat remains highly popular due to its extensive service integration, large user base and business-oriented features.

2.3.3 LINE

LINE is a widely used messaging application in Asia that provides text messaging, voice calls, video calls and media sharing. To improve communication security, LINE introduced Letter Sealing, an end-to-end encryption mechanism that protects messages during transmission.

Letter Sealing encrypts communication between users and helps prevent unauthorized access to message content. However, the availability and implementation of encryption may vary depending on platform configuration and application settings LY Corporation (2026). While LINE offers stronger privacy protection than some traditional messaging systems, researchers continue to recommend additional security controls such as secure authentication and attack monitoring mechanisms to strengthen overall communication security.

2.3.4 Proposed System: ChipChat

ChipChat is the proposed secure chat application developed in this project. Unlike many existing messaging applications that primarily focus on communication functionality, ChipChat was specifically designed to demonstrate secure communication using modern cryptographic technologies.

The system implements end-to-end encryption through a hybrid cryptographic architecture combining AES-GCM and RSA-OAEP. Messages are encrypted on the sender's device and can only be decrypted by the recipient. The server functions only as a relay mechanism and does not have access to plain text message contents.

In addition to encrypted communication, ChipChat incorporates several cybersecurity features including password hashing, encrypted user identifiers, SQL injection detection, cross-site scripting (XSS) detection, brute-force attack monitoring, replay attack detection and administrative security auditing. These features provide a more comprehensive demonstration of secure communication principles and cybersecurity practices compared to conventional messaging platforms.

Although ChipChat is currently a prototype developed for educational and research purposes, it demonstrates how cryptographic technologies and security monitoring mechanisms can be integrated into a secure communication platform.

2.4 Comparative Analysis

To better understand the strengths and limitations of existing messaging platforms, a comparative analysis was conducted between Telegram, WeChat, LINE and the proposed ChipChat system. The comparison focuses on communication functions, security features and system capabilities relevant to the objectives of this project.

Feature	Telegram	WeChat	LINE	ChipChat
Real-Time Messaging	✓	✓	✓	✓
File Sharing	✓	✓	✓	✓
End-to-End Encryption	Secret Chat Only	Limited	Letter Sealing	✓
AES Encryption	✓	✓	✓	✓
RSA Key Exchange	Not Publicly Disclosed	Not Publicly Disclosed	Not Publicly Disclosed	✓ RSA-OAEP
Admin Security Monitoring	✗	✗	✗	✓
Educational Demonstration	✗	✗	✗	✓

Table 2 Security Feature Comparison

Feature	Telegram	WeChat	LINE	ChipChat
Available on PC	Yes	Yes	Yes	Yes
Message Encrypted by Default	No	No	No	Yes
Send Text Messages	Yes	Yes	Yes	Yes
Send Images	Yes	Yes	Yes	Yes
Send Audio	Yes	Yes	Yes	Yes
Send Videos	Yes	Yes	Yes	Yes
User Registration	Yes	Yes	Yes	Yes
User Login	Yes	Yes	Yes	Yes
Password Recovery	Yes	Yes	Yes	Yes
Contact Search	Yes	Yes	Yes	Yes
Unread Message Indicator	Yes	Yes	Yes	Yes
Edit Messages	Yes	Limited	Yes	Yes
Delete Messages	Yes	Yes	Yes	Yes
Block Users	Yes	Yes	Yes	Yes
User Alias / Remark	No	Yes	Yes	Yes
Chat History Storage	Yes	Yes	Yes	Yes
Real-Time Messaging	Yes	Yes	Yes	Yes

Table 3 Functional Feature Comparison

Chapter 3 Methodology

3.1 Research / Development Approach

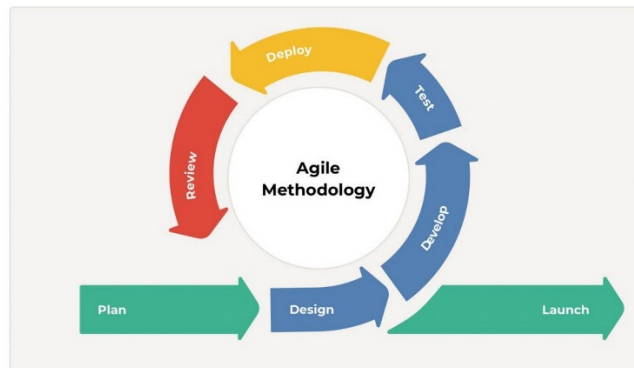


Figure 1 Agile Development Methodology

This project adopted the Agile development methodology. Agile was chosen because it allows features to be developed and improved gradually throughout the project. Unlike the traditional Waterfall model, Agile supports iterative development, where features can be designed, implemented, tested and improved in multiple cycles.

The development process was divided into several stages. During the early stages, the core communication functions such as user registration, login and real-time messaging were implemented. Subsequent development cycles focused on integrating security mechanisms including end-to-end encryption, password hashing, encrypted user identifiers and attack detection features. Additional improvements such as media sharing, user management and administrative monitoring functions were introduced in later stages.

The iterative nature of Agile allowed security issues, usability concerns and system bugs to be identified and corrected throughout development. This approach helped ensure that the final system met both functional and security requirements.

The development work was divided into several phases, where each phase focused on a specific function or improvement. In this project, the Agile process is broken down into five key stages:

1. Requirement Gathering and Planning

The first stage focused on identifying the functional and security requirements of the system. Existing messaging applications such as Telegram, WeChat and LINE were reviewed to understand their communication features and security capabilities. Based

on this analysis, several system requirements were identified, including user registration, login authentication, real-time messaging, media sharing and end-to-end encryption. Security requirements such as password protection, secure key exchange, attack detection and user privacy protection were also defined. These requirements were organized into a development plan and prioritized according to project objectives.

2. System Design

After gathering the requirements, the overall system architecture and database structure were designed. The system was developed using a client-server architecture consisting of a web-based frontend, a Flask backend server and a SQLite database. User interface layouts, system workflows and database relationships were planned during this stage. The security design also included the implementation of AES-GCM encryption for message protection, RSA-OAEP for secure key exchange, password hashing and encrypted user identifiers to enhance communication security.

3. Development and Implementation

The development stage involved building the main functions of the application through multiple iterations. Core features such as user registration, login, password recovery, real-time messaging and user management were implemented first. Security functions including end-to-end encryption, encrypted message transmission, SQL injection detection, cross-site scripting (XSS) detection, brute-force attack monitoring and replay attack detection were then integrated into the system. Additional features such as image sharing, audio recording, video sharing, user blocking and administrative monitoring functions were added during later development cycles.

4. Testing and Evaluation

Testing was conducted continuously throughout the development process to ensure system functionality and security. Functional testing was performed to verify features such as registration, login, messaging and media sharing. Security testing was conducted to evaluate the effectiveness of encryption, authentication and attack detection mechanisms. Simulated cybersecurity attacks including SQL injection, cross-site scripting (XSS), brute-force login attempts and replay attack detection were performed to assess the system's ability to identify and respond to security threats. Network traffic analysis using Wireshark was also conducted to verify encrypted

communication through HTTPS and TLS protocols.

5. Deployment and Review

After testing was completed, the application was deployed on the Render cloud hosting platform and connected to a public domain for accessibility. The deployed system was evaluated under real usage conditions to ensure stability and usability. Feedback obtained during testing was used to improve the user interface, system performance and security functions. The final version of the system successfully demonstrated secure communication through end-to-end encryption while providing additional cybersecurity protection and monitoring features.

Tools, Models and Frameworks

- Programming Languages and Development Tools
 - Python (Backend Development)
 - HTML, CSS and JavaScript (Frontend Development)
 - Bootstrap 5 (User Interface Design)
 - Flask and Flask-SocketIO (Web Application Framework)
 - SQLite (Database Management)
 - GitHub (Version Control)
 - Render (Cloud Deployment Platform)
- Cryptographic Technologies
 - AES-GCM (Message Encryption)
 - RSA-OAEP (Secure Key Exchange)
 - Password Hashing (Authentication Security)
 - Fernet Encryption (Phone Number Protection)

3.2 System Design

3.2.1 System Architecture

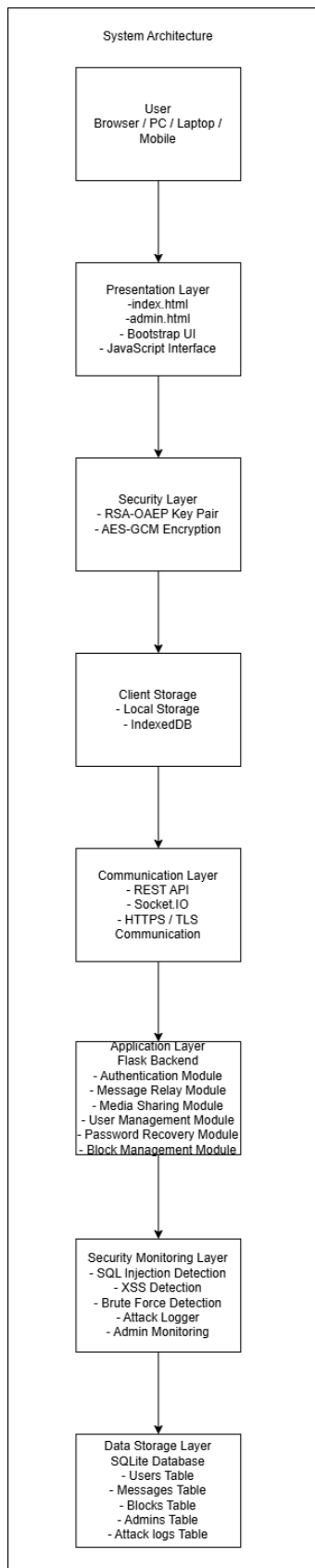


Figure 2 ChipChat System Architecture

The System Architecture shows the overall architecture of the proposed ChipChat secure messaging system. The system follows a layered client-server architecture consisting of presentation, security, communication, application, monitoring and database layers. The layered architecture separates system functions into different components, making the system easier to develop, manage and maintain.

Presentation Layer

The presentation layer acts as the main user interface of the system. This layer consists of the web pages developed using HTML, CSS, Bootstrap and JavaScript, including the main chat interface and the administrative dashboard. Users interact with the system through this layer to perform actions such as user registration, login, sending messages, media sharing and account management.

Security Layer

The security layer is responsible for protecting communication data and user privacy. AES-GCM encryption was implemented to encrypt message contents before transmission, while RSA-OAEP was used for secure key exchange between users. This layer ensures that messages remain encrypted during communication and can only be decrypted by authorized recipients.

Client Storage Layer

The client storage layer manages temporary data storage on the user's device. LocalStorage and IndexedDB were used to store user session information, encrypted keys and local chat history. This approach helps improve system responsiveness and allows encrypted message data to remain stored locally within the client environment.

Communication Layer

The communication layer manages data transmission between the client and the backend server. REST API requests and Socket.IO real-time communication were used to support user authentication, messaging and system interactions. HTTPS and TLS protocols were also utilized to provide secure communication during data transmission over the network.

Application Layer

The application layer contains the core backend functions of the system and was developed using Python Flask and Flask-SocketIO. This layer handles user authentication, message relay, user management, password recovery and media sharing functions. The backend server processes incoming requests, manages encrypted communication data and coordinates interactions between users and the database.

Security Monitoring Layer

The security monitoring layer was designed to detect and log suspicious activities within the system. Several security controls were added, including SQL injection detection, cross-site scripting (XSS) detection, brute-force login detection and replay attack detection. Security logs and attack information were also recorded and displayed through the administrative monitoring dashboard.

Data Storage Layer

The data storage layer uses a SQLite database to store application data. Several database tables were implemented, including users, messages, blocks, admins and attack logs. This layer is responsible for maintaining user information, encrypted communication records, account management data and security monitoring logs required by the system.

3.2.2 Flowchart

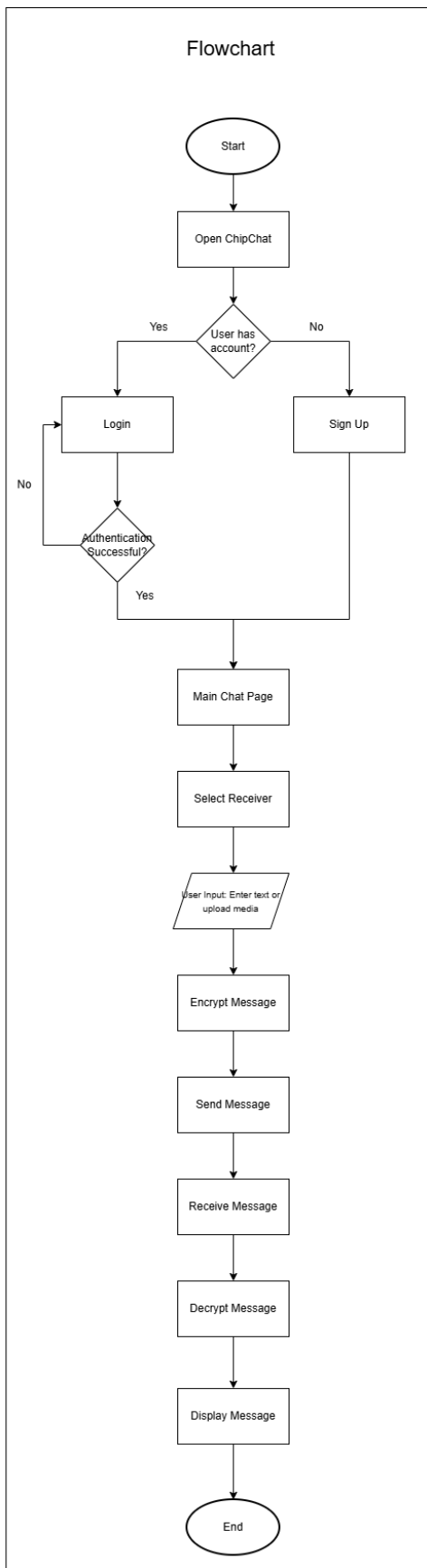


Figure 3 ChipChat System Flowchart

The System Flowchart shows the overall workflow of the ChipChat application from user access to secure message delivery. The process begins when the user opens the ChipChat application. The system first checks whether the user already has an account. Existing users proceed to the login page, while new users are required to complete the registration process before accessing the system.

For existing users, authentication is performed using the registered phone number and password. If authentication fails, the user is redirected back to the login page and prompted to try again. Once authentication is successful, the user is granted access to the main chat interface.

After entering the main chat page, the user selects a receiver from the available contact list and enters a text message or uploads media content such as images, audio or video files. Before transmission, the message is encrypted using the AES-GCM encryption algorithm to protect the confidentiality of communication data.

The encrypted message is then transmitted through the server and delivered to the intended recipient. Upon receiving the message, the recipient's device decrypts the message and converts the encrypted ciphertext back into readable plaintext. Finally, the decrypted message is displayed on the recipient's chat interface, allowing secure communication between both users.

This workflow ensures that messages remain protected during transmission and can only be accessed by authorized users, supporting the implementation of secure communication within the ChipChat system.

3.2.3 ERD

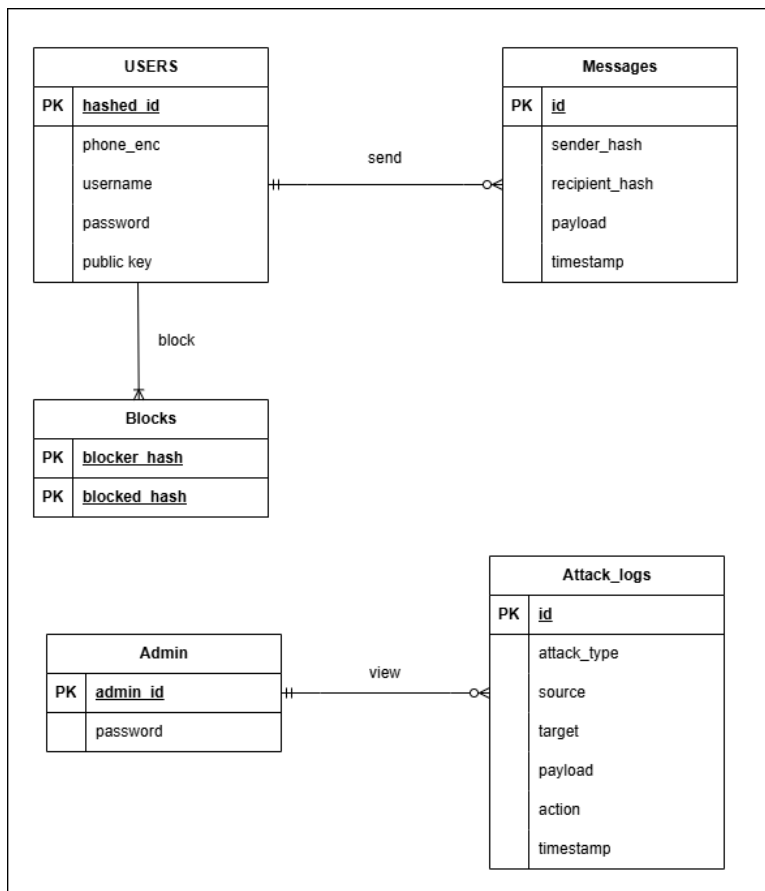


Figure 4 Entity Relationship Diagram

The Entity Relationship Diagram (ERD) shows the database structure of ChipChat. The database consists of five main entities: Users, Messages, Blocks, Admin and Attack_logs.

The Users entity stores user account information including the hashed user identifier, encrypted phone number, username, password hash and public key. Messages entity stores encrypted communication data exchanged between users. Each message record contains the sender identifier, recipient identifier, encrypted payload and timestamp information.

The Blocks entity manages user blocking relationships and uses a composite primary key consisting of blocker_hash and blocked_hash. This structure prevents duplicate blocking records while maintaining user privacy.

The Admin entity stores administrator credentials used to access the monitoring dashboard. Administrators are responsible for reviewing system activities and security events.

The Attack_logs entity records security-related events detected by the system. Information such as attack type, source, target, payload, action taken and timestamp is stored to support

security monitoring and incident analysis.

User remarks and aliases are stored locally using IndexedDB rather than the SQLite database. This design allows each user to manage personal display names without exposing private contact preferences to the server.

Overall, the database design supports secure user management, encrypted communication, access control and cybersecurity monitoring within the ChipChat application.

3.2.4 Use Case Diagram

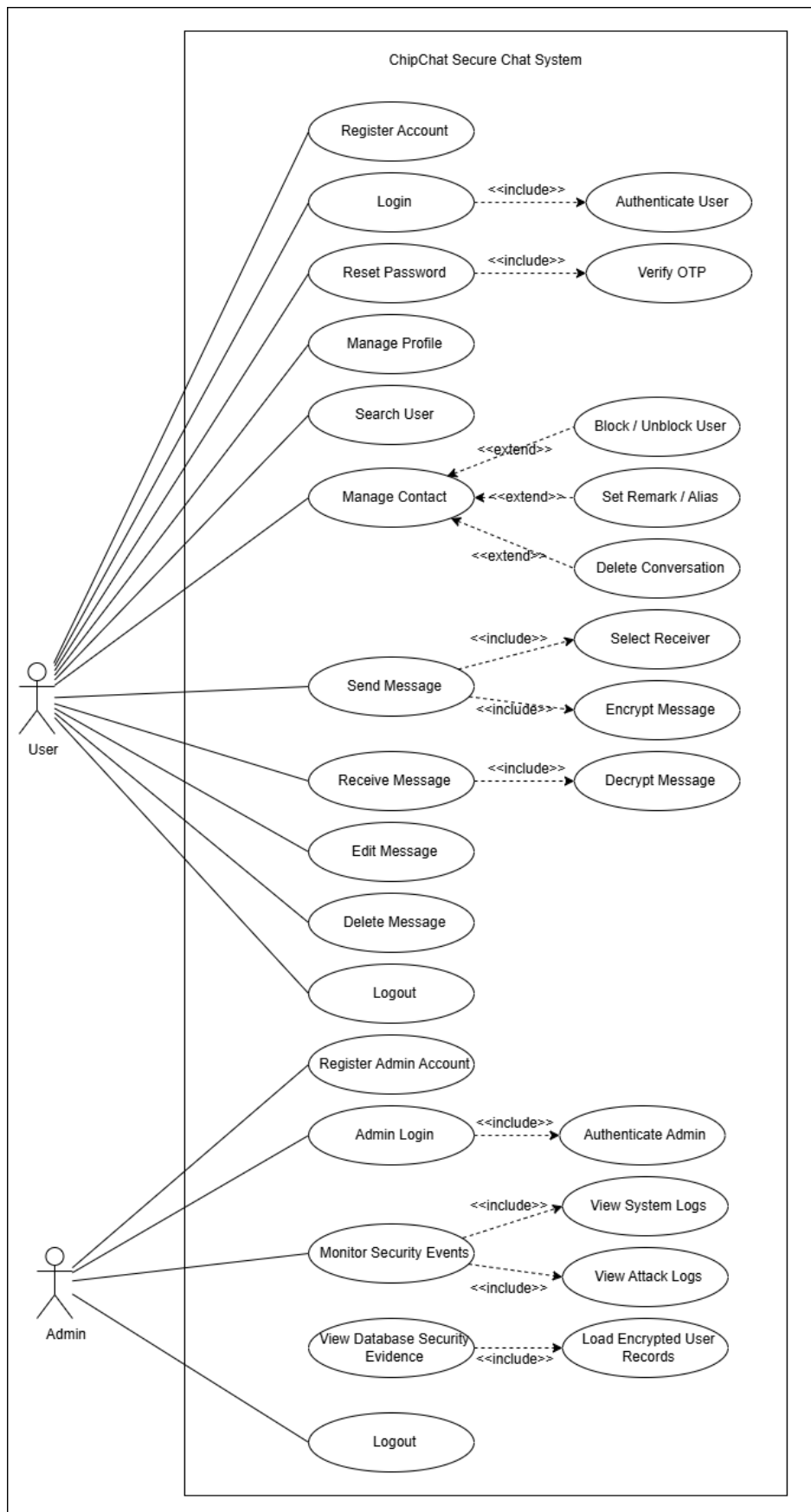


Figure 5 Use Case Diagram

The Use Case Diagram shows the main interactions between users, administrators and the ChipChat secure chat system. Two actors are involved in the system: User and Admin.

The User actor represents the normal application user. Users can register an account, log in, reset passwords, manage profiles, search for other users, manage contacts and exchange encrypted messages. Additional functions such as blocking users, setting aliases, deleting conversations, editing messages and deleting messages are also supported. Before a message is transmitted, the system performs receiver selection and message encryption to ensure secure communication. Received messages are decrypted before being displayed to the recipient.

The Admin actor represents the system administrator who is responsible for monitoring security-related activities. Administrators can register and log in to the administration panel, monitor security events, review system logs, inspect attack logs and view encrypted user records stored within the database.

Overall, the Use Case Diagram provides an overview of the functional requirements of the ChipChat system and demonstrates how both users and administrators interact with the application to support secure communication and system monitoring.

Chapter 4 Implementation

4.1 System Architecture Overview

The ChipChat application was implemented using a layered client-server architecture. The system consists of a web-based frontend developed using HTML, CSS, JavaScript and Bootstrap, a backend server developed using Flask and Flask-SocketIO, and a SQLite database for data storage.

The frontend is responsible for handling user interaction, including account registration, login, messaging and media sharing. The backend processes user requests, manages authentication, handles encrypted message transmission and performs security monitoring functions. Communication between clients and the server is secured using HTTPS and TLS protocols.

To protect communication privacy, RSA-OAEP was implemented for secure key exchange, while AES-GCM was used for message encryption. Additional security features such as SQL injection detection, XSS detection, brute-force attack monitoring and replay attack detection were integrated into the system.

Component	Technology Used
Frontend	HTML, CSS, JavaScript, Bootstrap 5
Backend	Python Flask
Real-Time Communication	Flask-SocketIO
Database	SQLite
Encryption	AES-GCM, RSA-OAEP
Security Testing	Wireshark
Version Control	GitHub
Cloud Deployment	Render
Development IDE	Visual Studio Code
Web Browser	Google Chrome

Table 4 Development Environment

4.2 User Interface Implementation

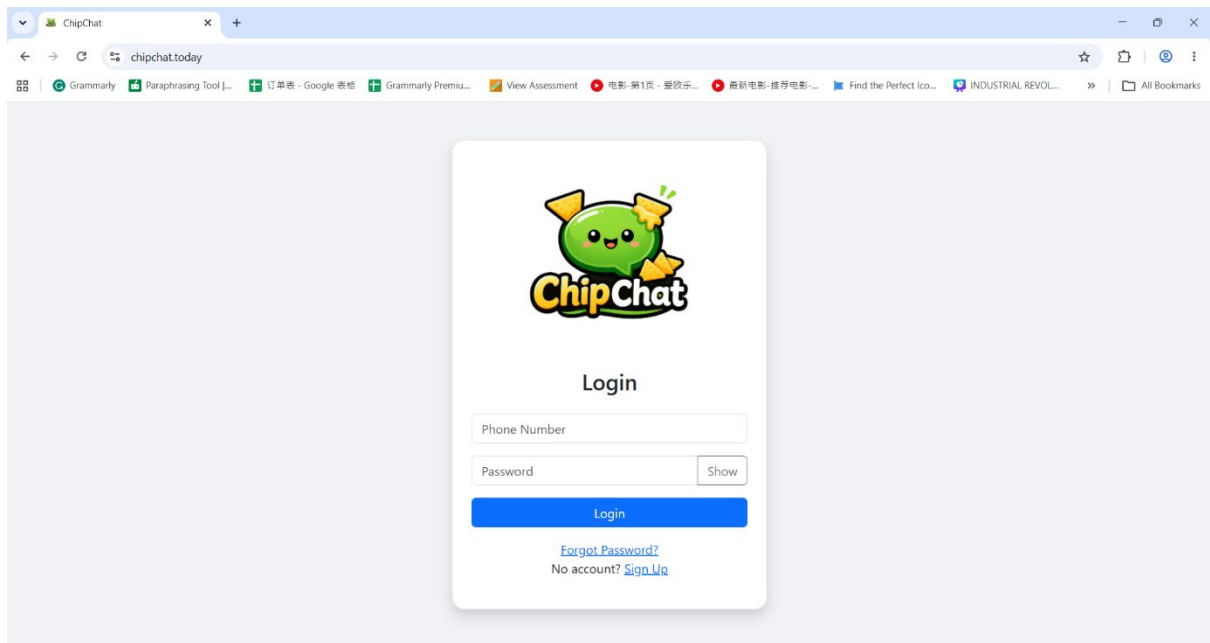


Figure 6 Login Page

The login page allows registered users to access the ChipChat system using their phone number and password. The system verifies user credentials before granting access to the main chat interface.

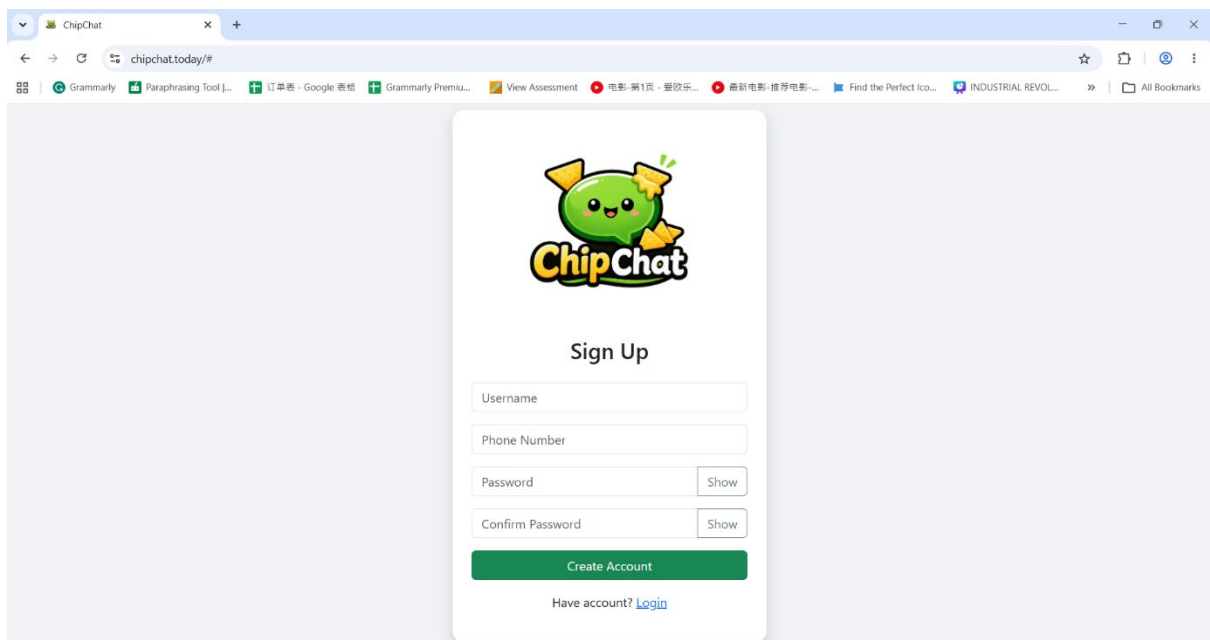


Figure 7 Registration Page

The registration page allows new users to create an account by entering a username, phone number and password. User information is validated and securely stored before access is granted to the system.

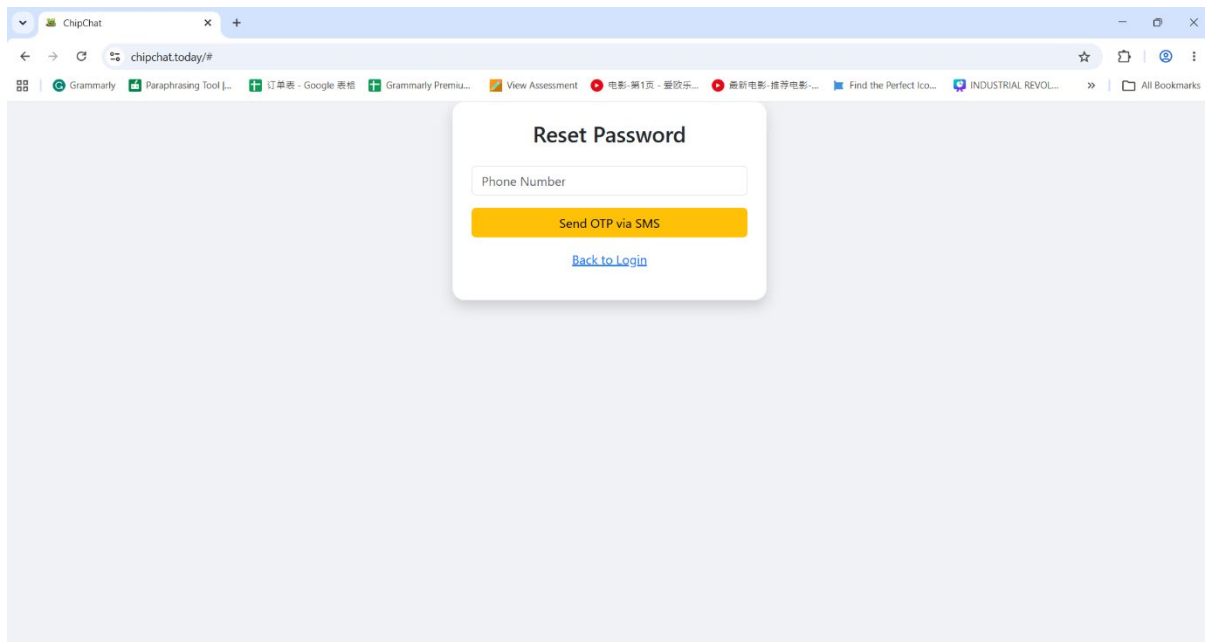


Figure 8 Password Recovery Page

The password recovery page allows users to reset their account password by entering their registered phone number. An OTP verification process is used to validate the user's identity before allowing password recovery.

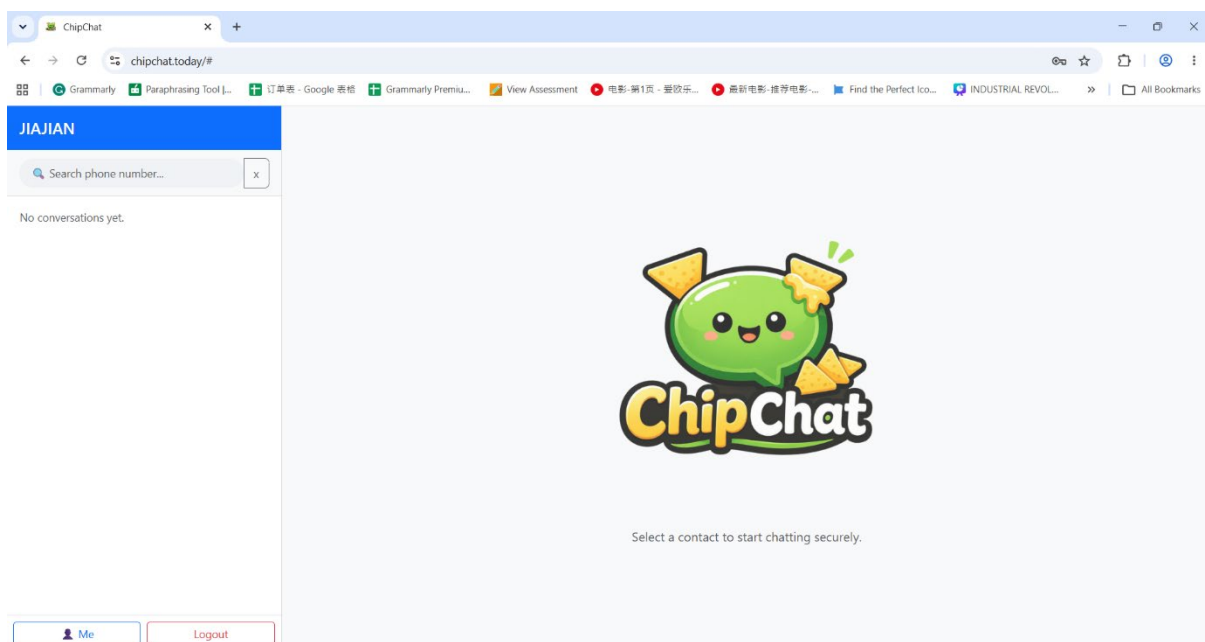


Figure 9 Main Chat Page

The main chat page is displayed after a user successfully logs into the system. Users can search for contacts using phone numbers, access their profile and manage conversations from this page. When no contact is selected, the system displays a logo screen and prompts the user to start a secure chat.

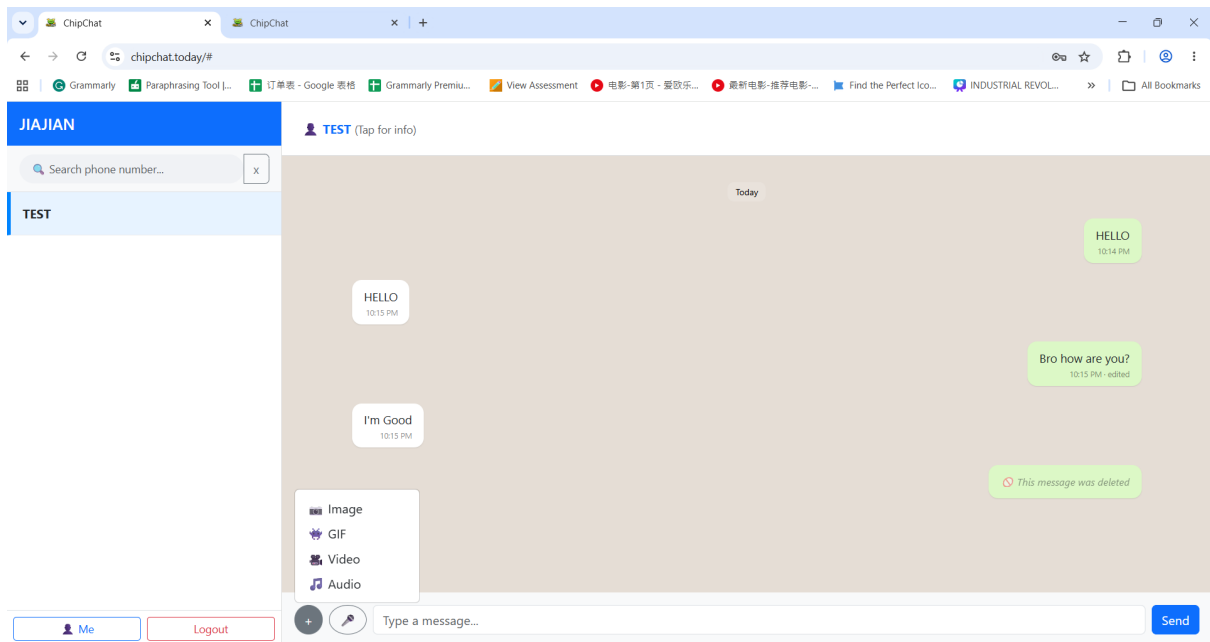


Figure 10 Main Chat Interface

The main chat interface allows users to select a receiver, view conversations and exchange messages in real time. The interface supports sent and received messages, edited messages, deleted messages and date separators. Users can also send different media types such as voice messages, images, GIFs, videos and audio files through the attachment menu.

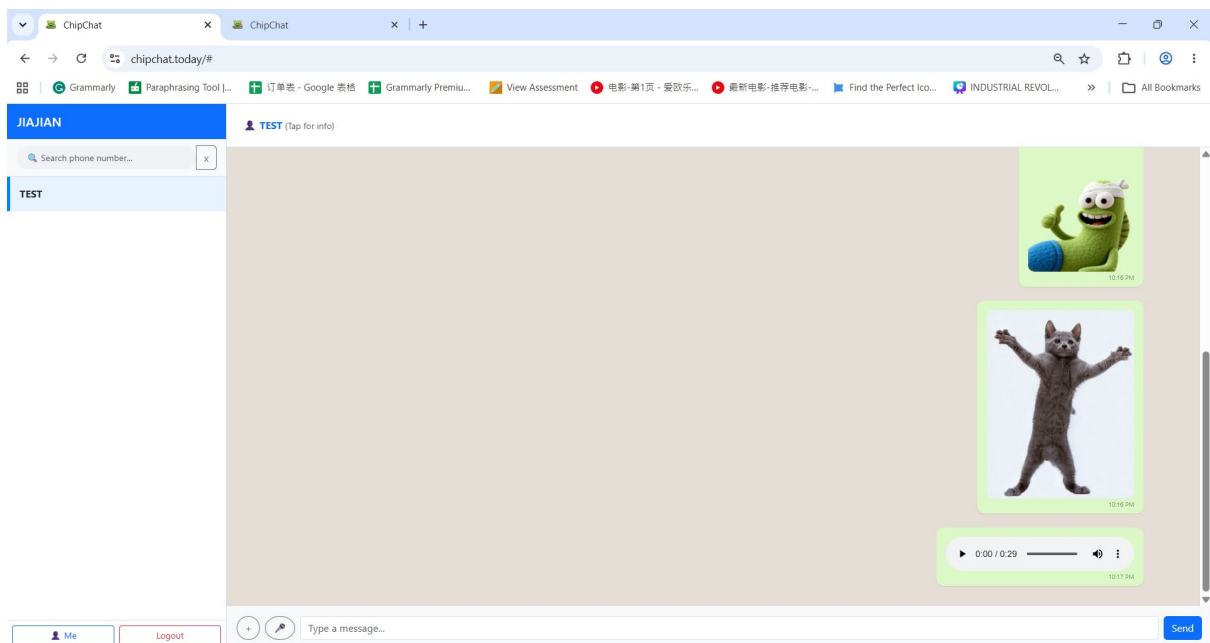


Figure 11 Media Sharing Sender Side

The media sharing feature allows users to send different types of media files, including images, GIFs and audio messages. Uploaded media is displayed within the chat interface and transmitted to the selected recipient in real time.

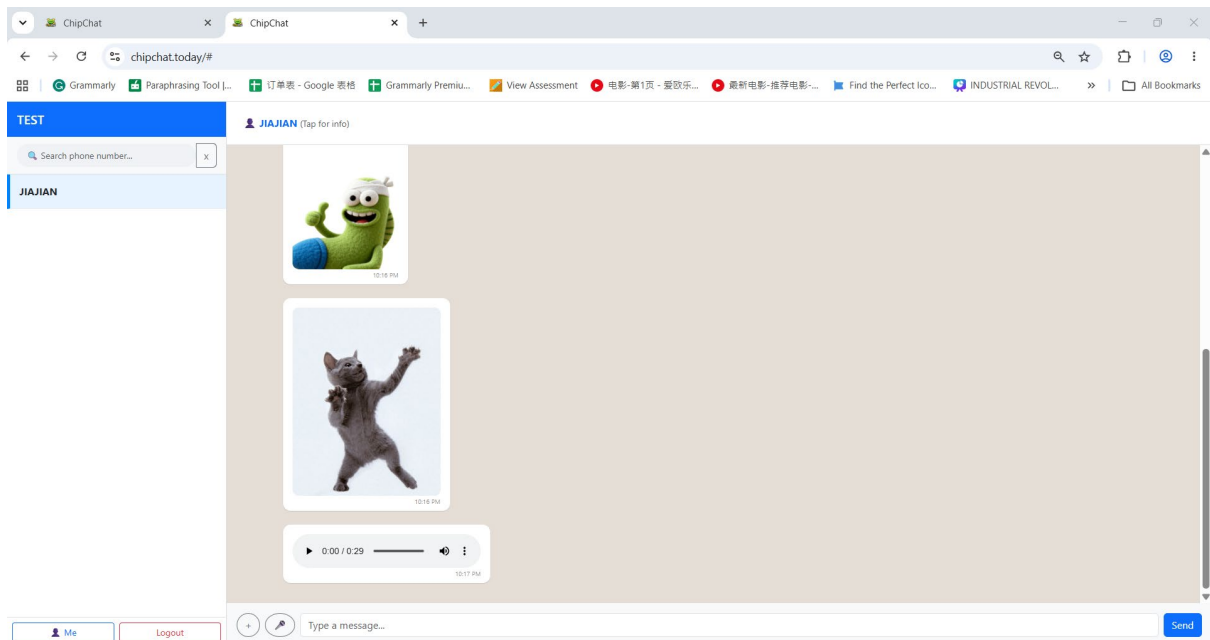


Figure 12 Media Reception Receiver Side

The receiver is able to view and access shared media directly within the chat interface. Images, GIFs and audio messages are displayed in their original format, allowing users to communicate using both text and multimedia content.

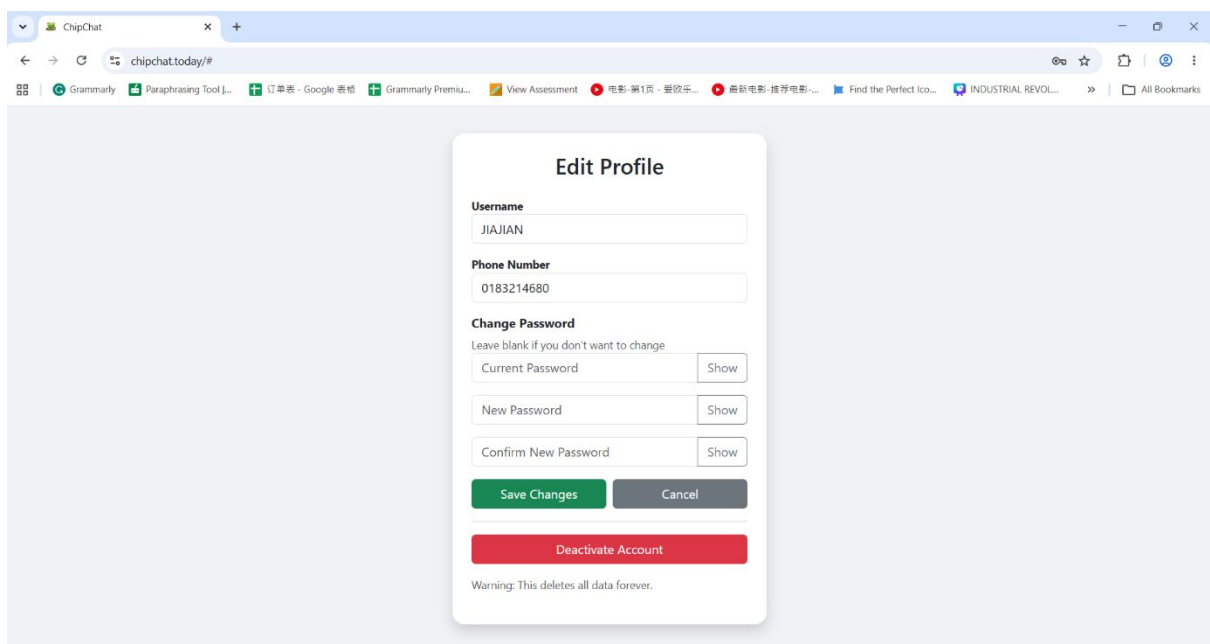


Figure 13 Edit Profile Page

The edit profile page allows users to update their username, phone number and password. Password changes require verification of the current password before a new password can be set. Users may also deactivate their account through this page.

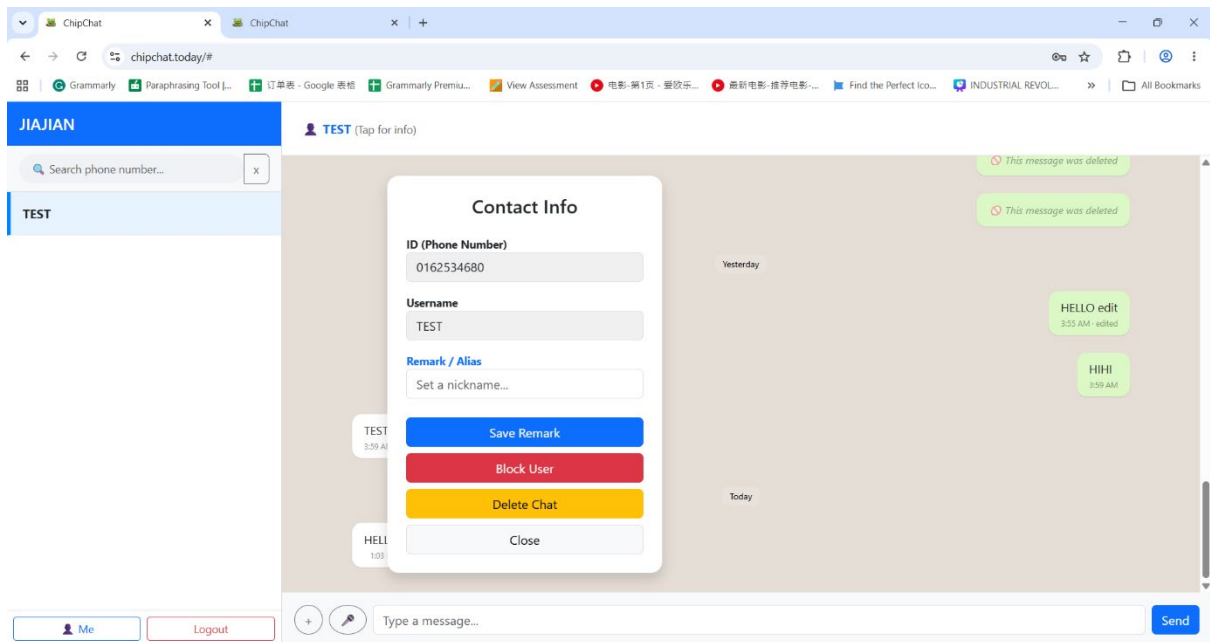


Figure 14 Contact Information Page

The contact information page provides several contact management functions. Users can assign a remark or alias to a recipient, block or unblock users, delete chat history and view basic contact information. These features help users manage their conversations more effectively.

4.3 Admin Interface Implementation

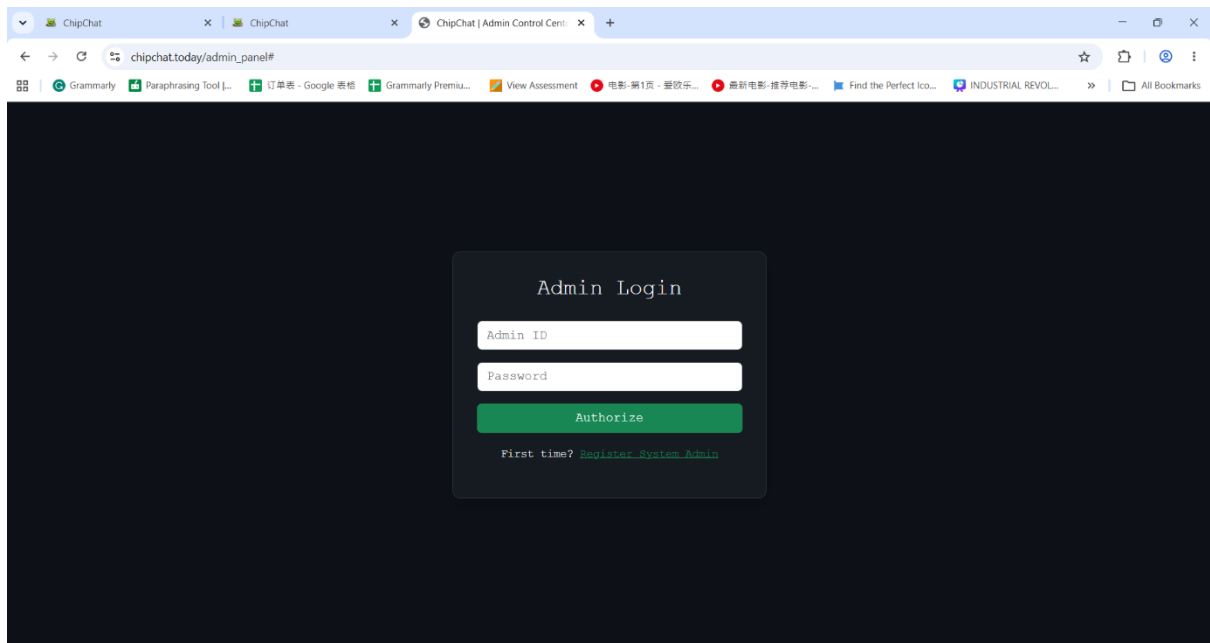


Figure 15 Admin Login Page

The admin login page allows authorized administrators to access the ChipChat administrative panel using an admin ID and password. Only authenticated administrators are granted access to system monitoring and management functions.

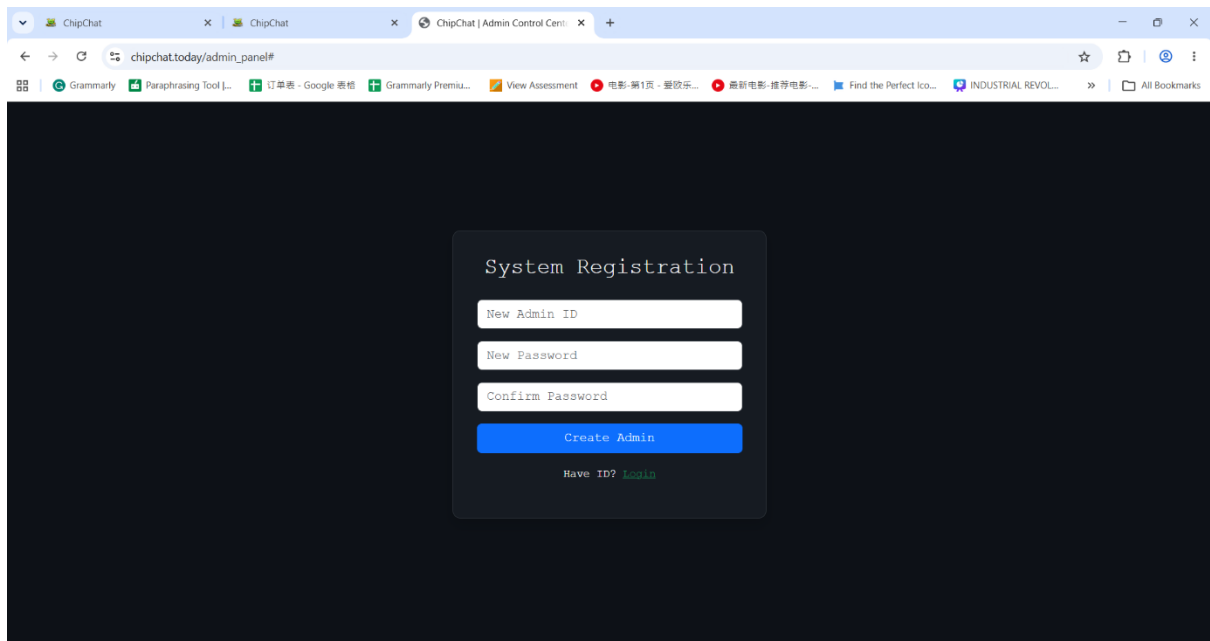


Figure 16 Admin Registration Page

The admin registration page allows the creation of new administrator accounts. Administrators are required to provide a valid admin ID and password, which must be confirmed before the account is successfully created.

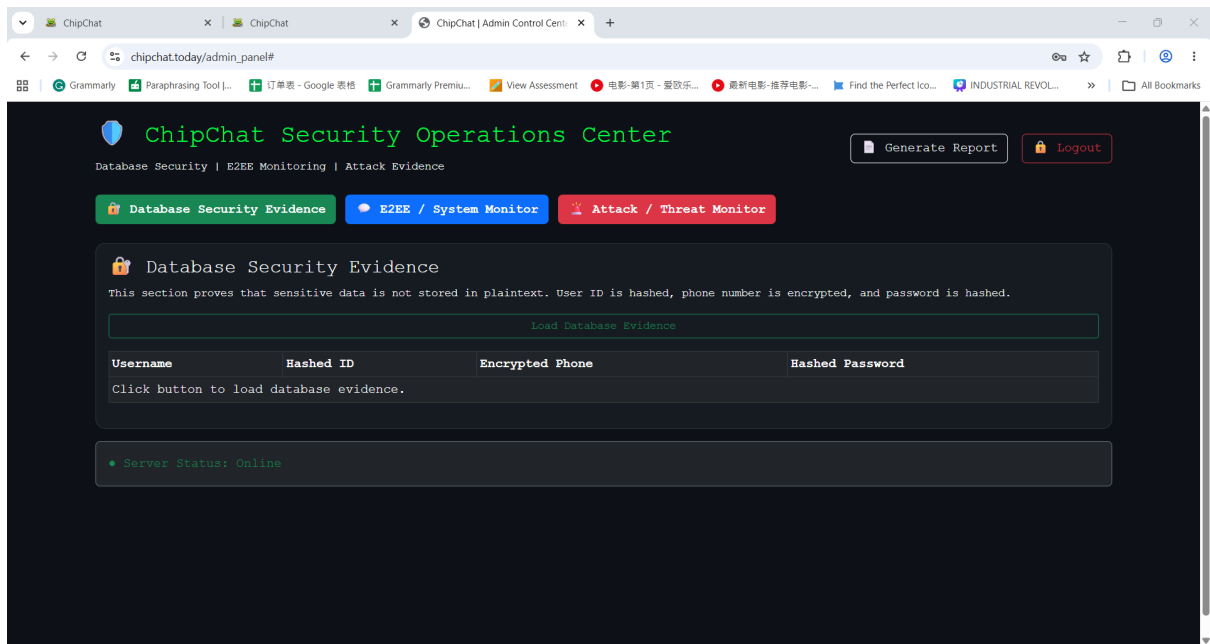


Figure 17 Security Operations Center Dashboard

The Security Operations Center serves as the main administrative dashboard of the ChipChat system. Administrators can access database security evidence, monitor encrypted communication activities, review attack logs and generate reports through this interface.

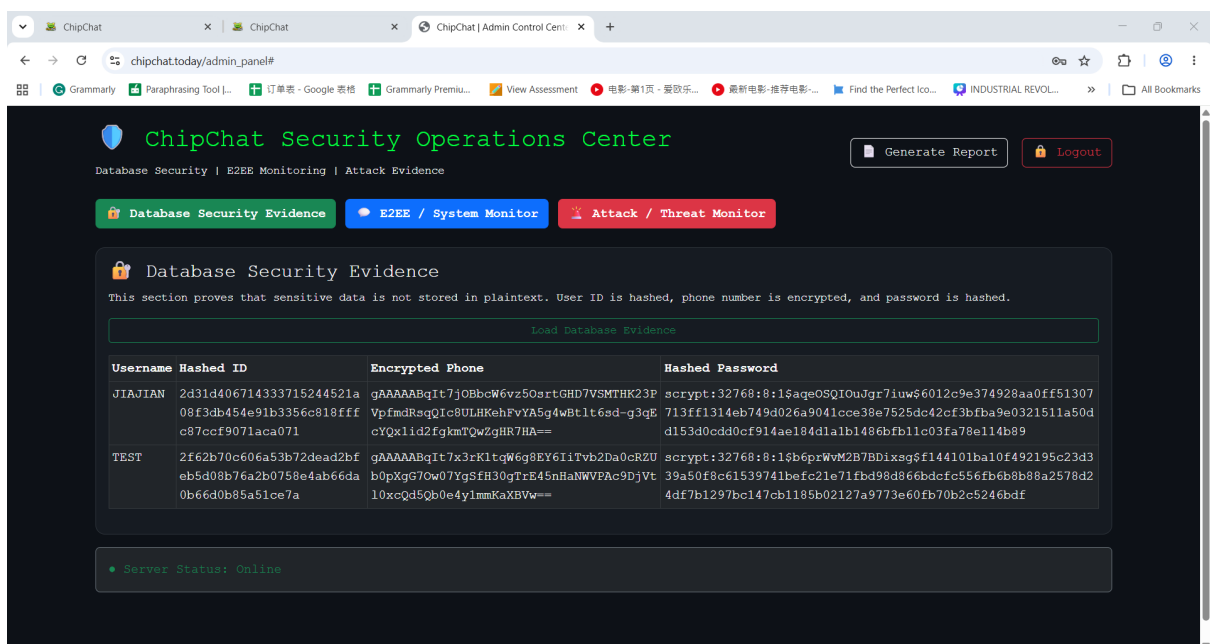


Figure 18 Database Security Evidence

The Database Security Evidence page demonstrates that sensitive user information is not stored in plaintext. User identifiers are hashed, phone numbers are encrypted and passwords are stored as hashed values to improve data security and privacy.

4.4 Security Feature Implementation

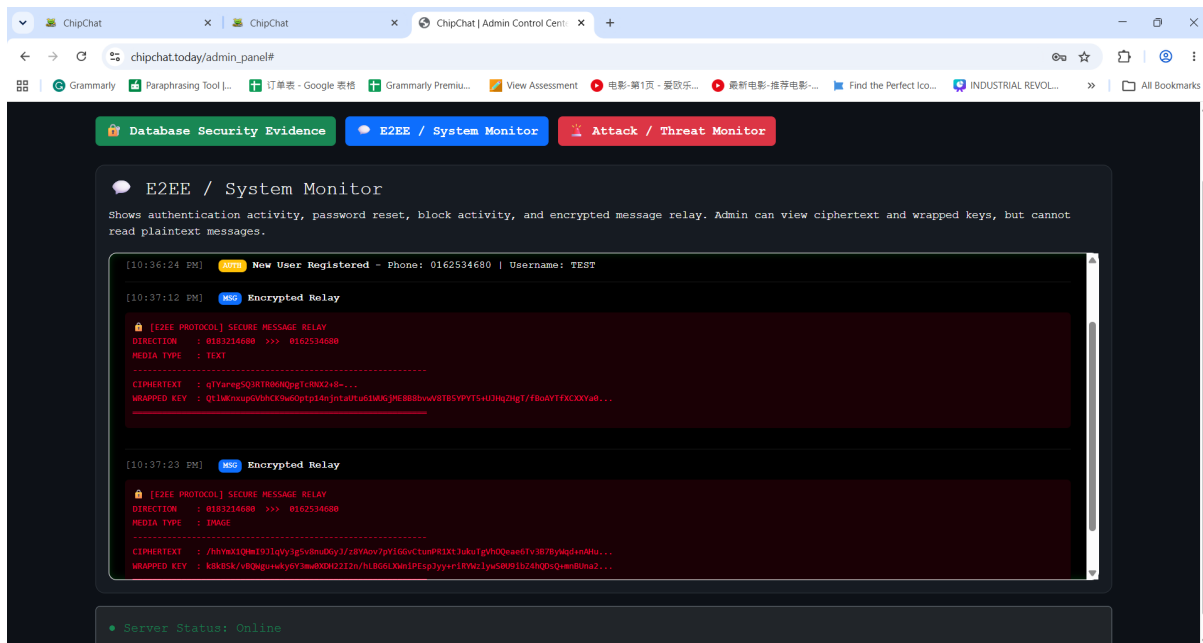


Figure 19 E2EE Monitoring Panel

The E2EE Monitoring Panel allows administrators to monitor encrypted communication activities within the system. Administrators can view message relay information, ciphertext and wrapped encryption keys for auditing purposes. However, the actual message content remains unreadable, demonstrating the implementation of end-to-end encryption in the ChipChat system.

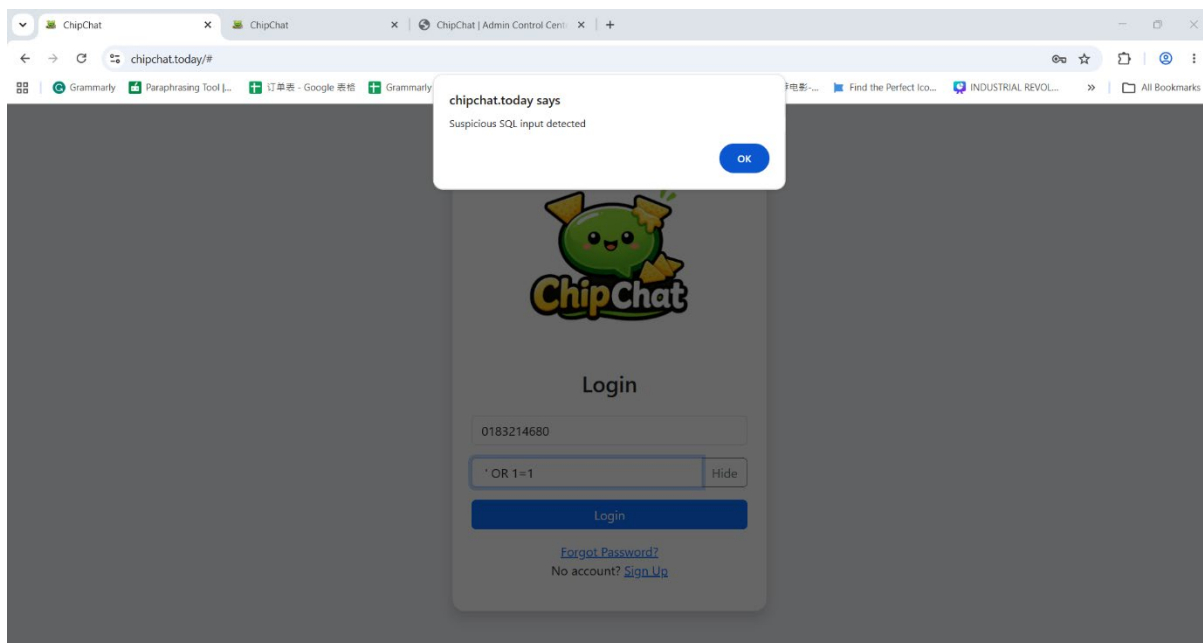


Figure 20 SQL Injection Detection

The system detects suspicious SQL injection patterns entered by users and immediately

blocks the request. A warning message is displayed to notify the user that potentially malicious input has been identified.

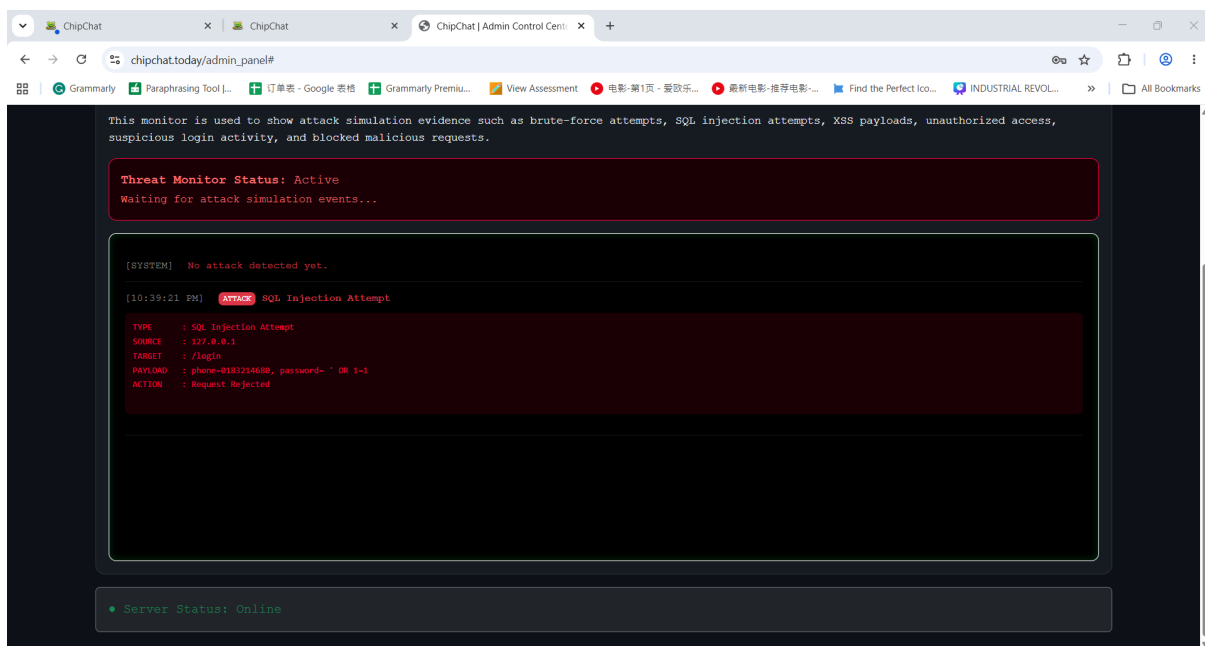


Figure 21 SQL Injection Attack Log

Detected SQL injection attempts are recorded in the Attack and Threat Monitoring panel. The log includes information such as the attack type, source, target, payload and action taken by the system for security monitoring purposes.

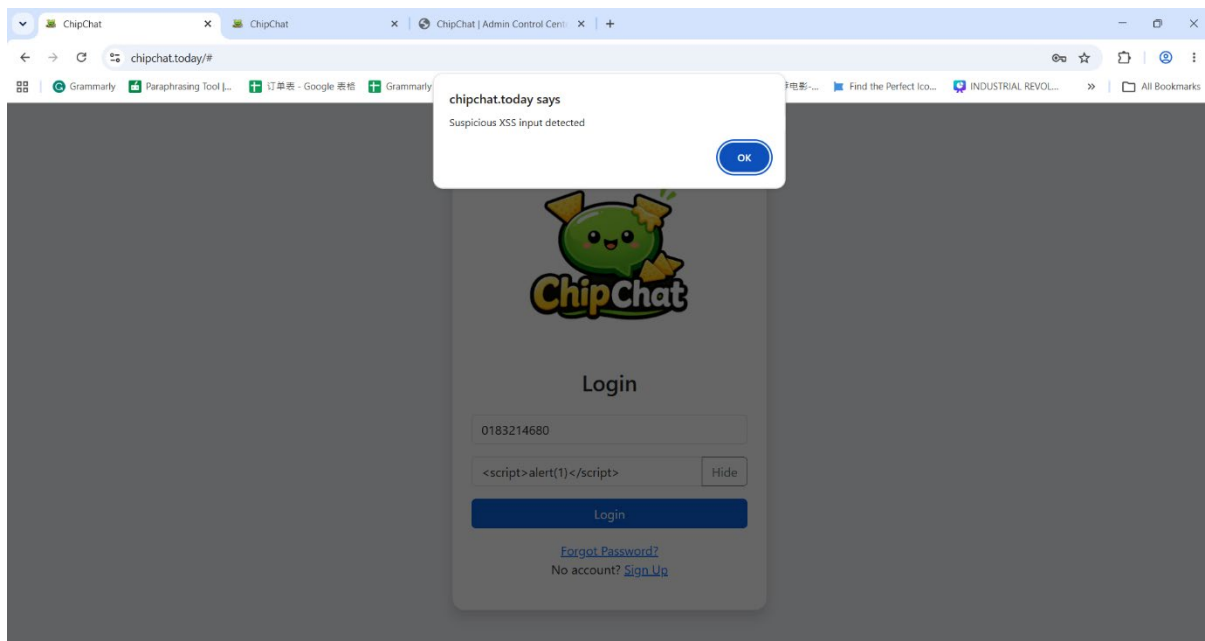


Figure 22 XSS Detection

The system detects suspicious Cross-Site Scripting (XSS) payloads entered by users and blocks the request before it can be processed. A warning message is displayed to indicate that

malicious input has been detected.

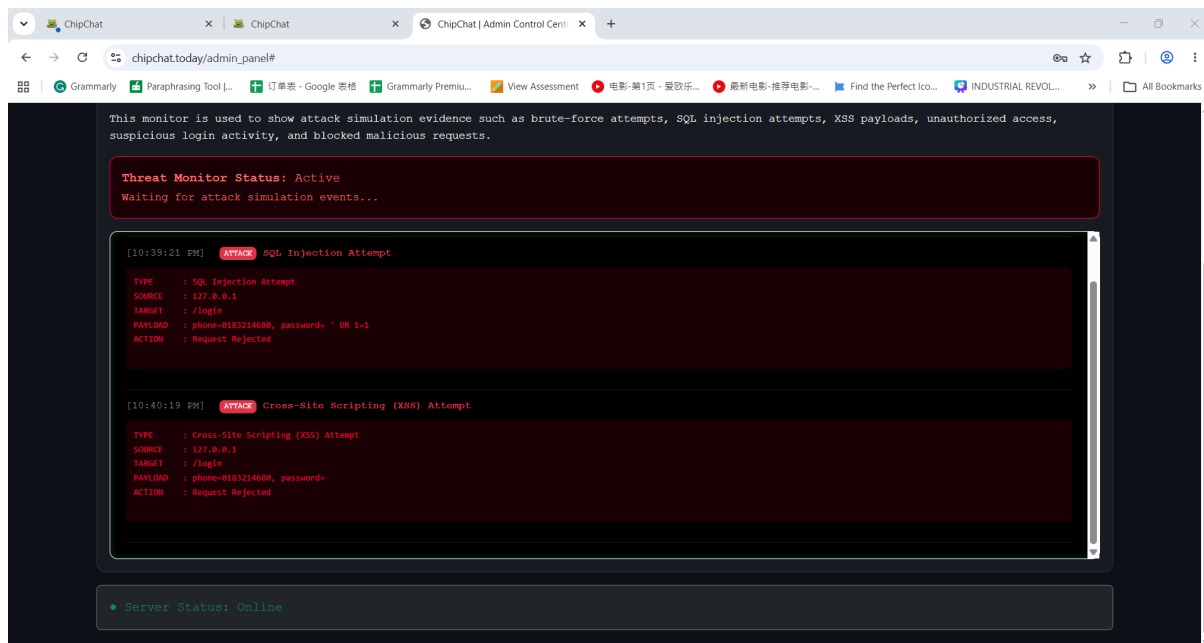


Figure 23 XSS Attack Log

Detected XSS attempts are recorded in the Attack and Threat Monitoring panel. The system logs the attack details and rejects the request to prevent malicious script execution within the application.

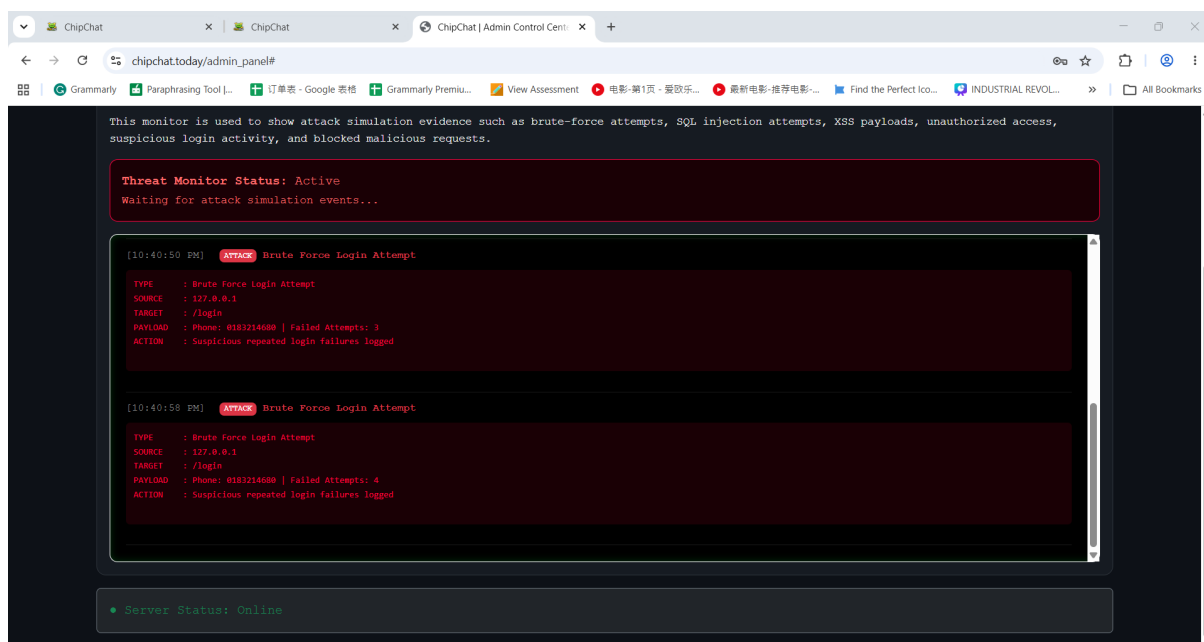


Figure 24 Brute Force Attack Log

The system monitors repeated failed login attempts and records potential brute-force attacks in the Attack and Threat Monitoring panel. Suspicious login activities are logged for administrative review and security analysis.

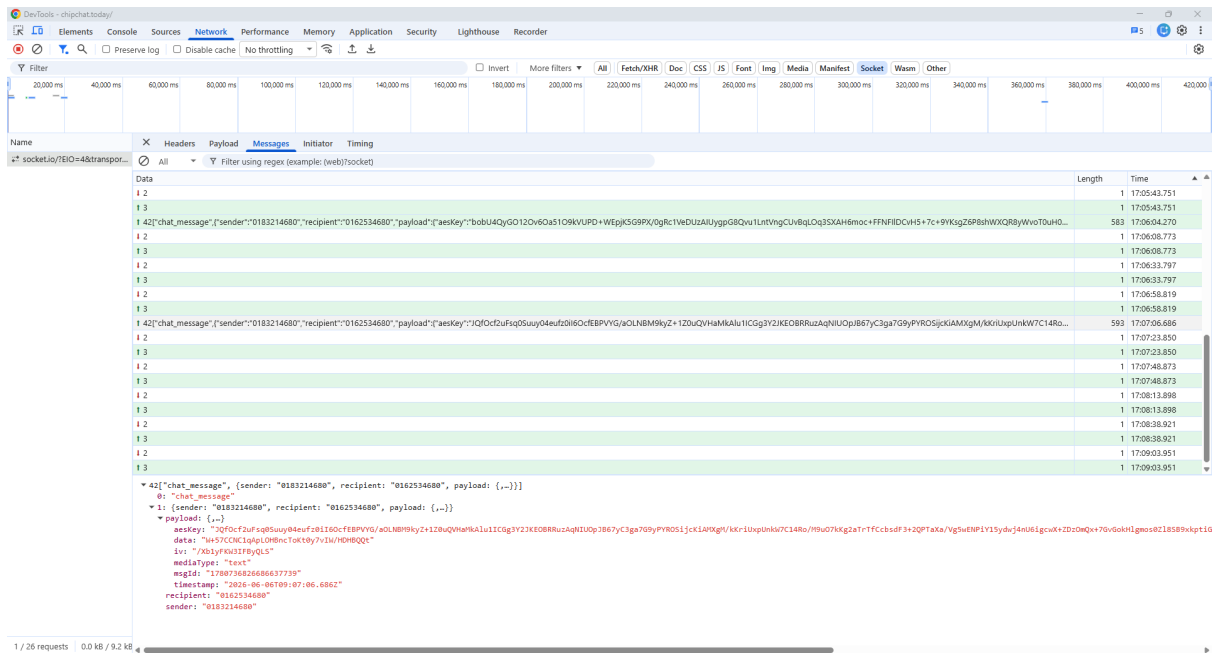


Figure 25 Replay Attack Message Capture

The replay attack test began by capturing an encrypted Socket.IO message packet transmitted between users. The packet contains a unique message identifier (msgId) that is later used by the system to verify message authenticity.

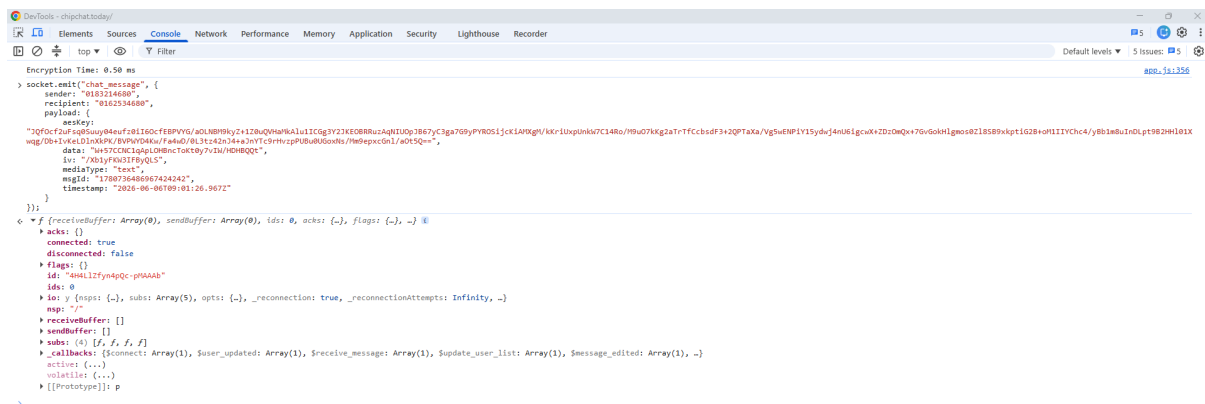


Figure 26 Replay Attack Simulation

The captured encrypted message was retransmitted through the browser console using the same message identifier. This simulation demonstrates how an attacker may attempt to resend previously transmitted communication data.

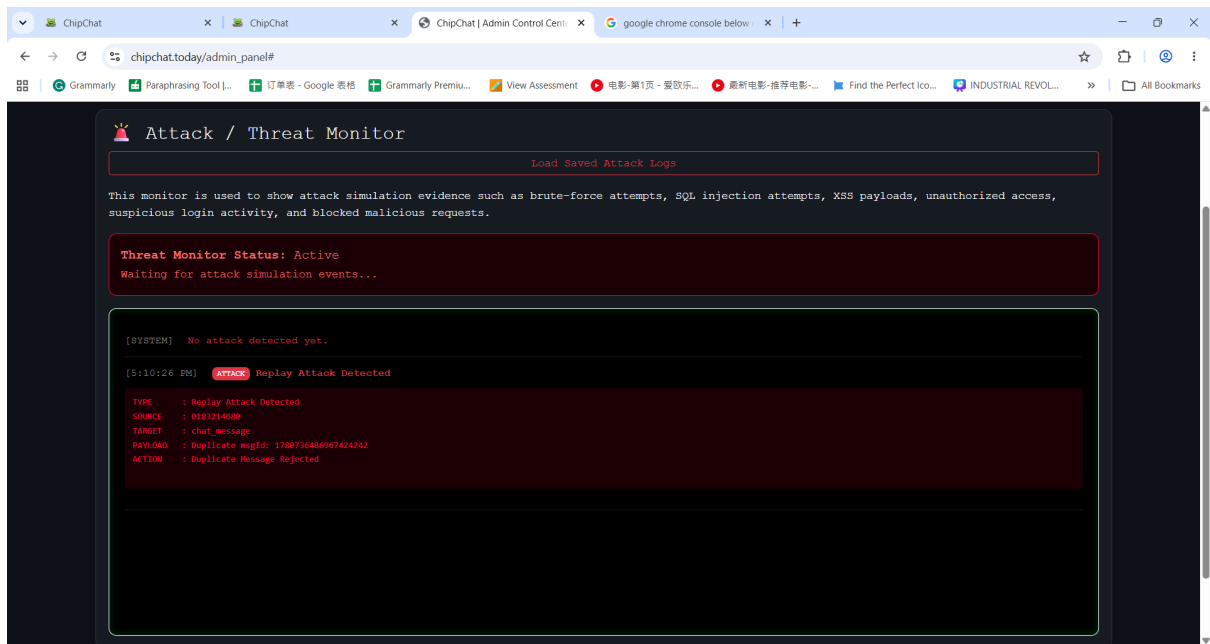


Figure 27 Replay Attack Detection Log

The system successfully detected the duplicated message identifier and classified the event as a replay attack. The message was rejected and the attack details were recorded in the monitoring dashboard for security analysis.

4.5 Network Security Verification

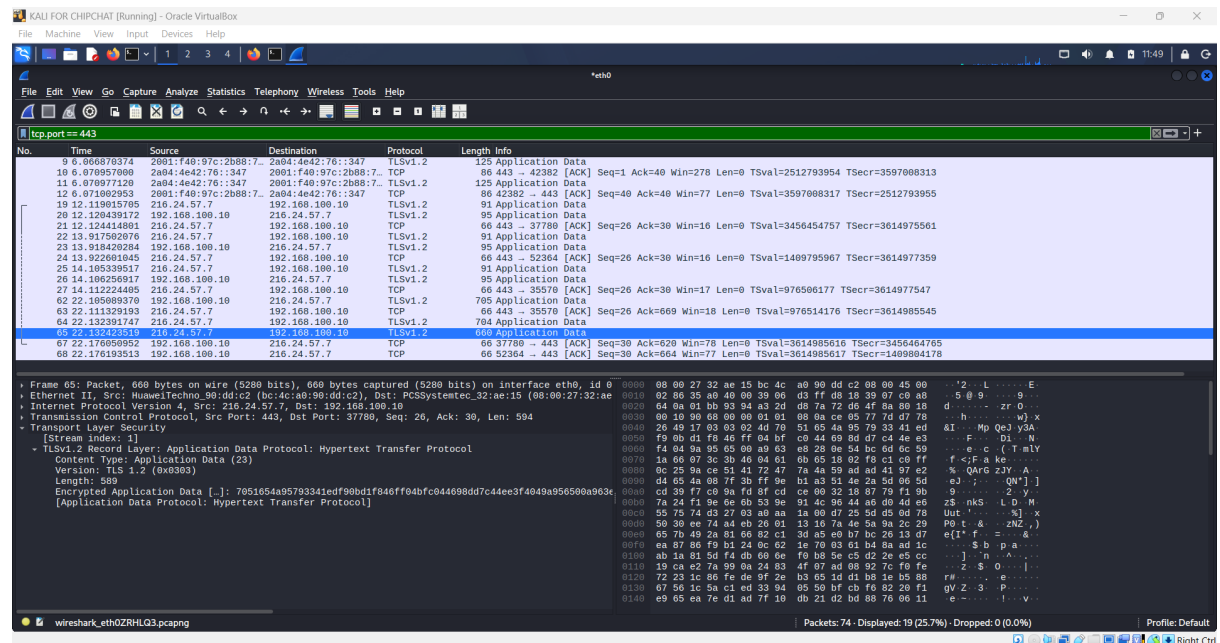


Figure 28 Wireshark TLS Capture

The Wireshark capture confirms that communication is protected using TLS 1.2 over HTTPS. Message contents are displayed as encrypted application data rather than plaintext, indicating that data is securely transmitted across the network.

Chapter 5 Discussion

5.1 Discussion of Objective 1

The first objective of this project was to develop a secure chat application using RSA-OAEP and AES-GCM cryptographic algorithms with end-to-end encryption. This objective was successfully achieved through the development of the ChipChat application.

The implemented system allows users to exchange encrypted messages and media files through a secure communication channel. AES-GCM was used to encrypt message contents, while RSA-OAEP was used to protect the encryption key before transmission. As a result, messages remained encrypted throughout transmission and could only be decrypted by the intended recipient.

The successful implementation of these cryptographic mechanisms demonstrates that secure communication can be achieved within a web-based messaging environment. Therefore, the first objective was fully achieved.



Figure 29 E2EE Message Relay Evidence

5.2 Discussion of Objective 2

The second objective focused on ensuring message confidentiality, integrity and authenticity. The developed system successfully addressed these requirements through the implementation of multiple security mechanisms.

Message confidentiality was achieved through end-to-end encryption, preventing unauthorized parties from accessing message contents. Message integrity was supported through AES-GCM encryption, which helps detect unauthorized modifications to encrypted data. User authentication was implemented through secure login procedures, password hashing and account verification processes.

In addition to encryption, the system incorporated cybersecurity protection mechanisms including SQL injection detection, cross-site scripting (XSS) detection and brute-force attack monitoring. These features provided additional protection beyond message encryption and improved the overall security of the application. Therefore, the second objective was successfully achieved.

The implementation of AES-GCM, RSA-OAEP, password hashing, attack detection mechanisms and TLS communication successfully contributed to the confidentiality, integrity and authenticity of user communications.



Figure 30 Discussion of SQL Injection Attack Log



Figure 31 Discussion of XSS Attack Log



Figure 32 Discussion of Brute Force Attack Log



Figure 33 Discussion of Replay Attack Detection Log

5.3 Discussion of Objective 3

The third objective aimed to evaluate the performance and usability of the developed system. Testing results indicated that encrypted messages could be processed and delivered with minimal delay, allowing users to communicate in real time while maintaining security.

Encryption performance testing demonstrated that AES-GCM encryption could be performed efficiently within the browser environment. Message delivery testing showed that communication between users remained responsive despite the additional encryption processes. Furthermore, the web-based interface provided a simple and user-friendly environment for registration, messaging and account management.

The results suggest that strong security can be implemented without significantly affecting system usability or communication efficiency. Therefore, the third objective was achieved successfully.

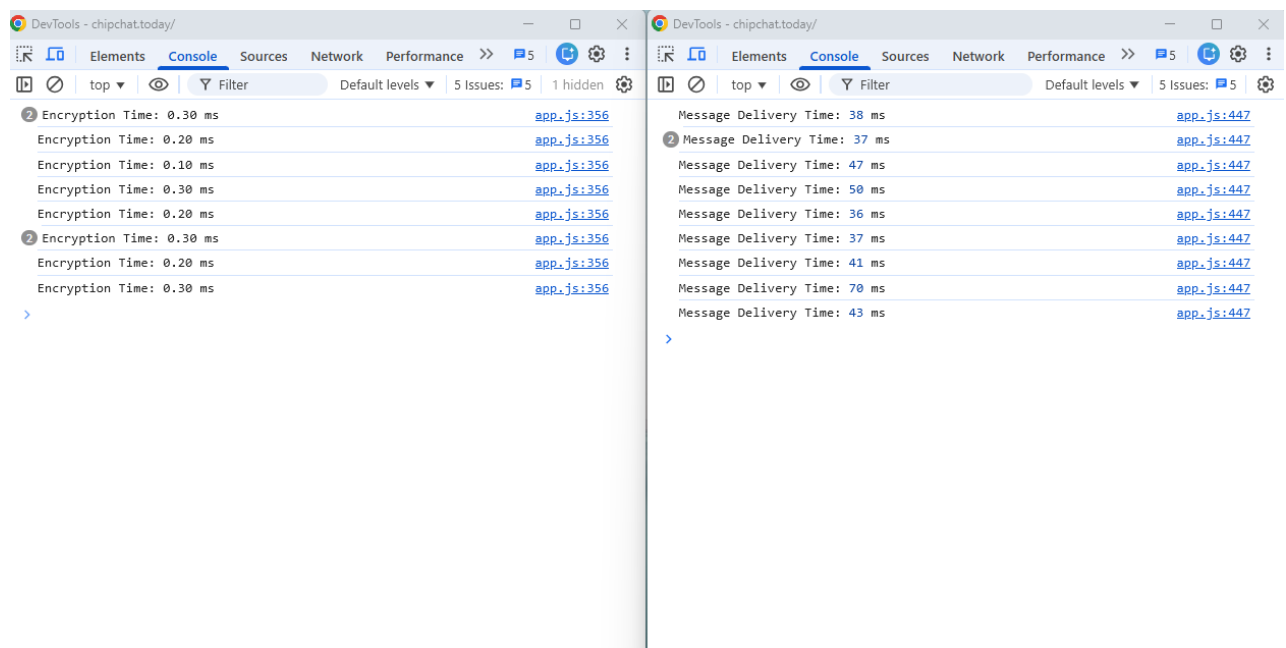


Figure 34 Encryption and Delivery Time Testing

Figure 30 shows the performance testing results of the ChipChat application. The encryption process recorded execution times between 0.10 ms and 0.30 ms, indicating that AES-GCM encryption can be performed efficiently without causing noticeable delays to users. Message delivery times ranged from 36 ms to 70 ms during testing, demonstrating that encrypted messages can be transmitted and received in near real-time. These results suggest that the implemented security mechanisms provide strong protection while maintaining acceptable communication performance and user experience.

Test	Result
Encryption Time	0.10 ms – 0.30 ms
Average Encryption Time	0.25 ms
Delivery Time	36 ms – 70 ms
Average Delivery Time	43.6 ms

Table 5 Performance Testing Results

5.4 Strengths of the Proposed System

The developed ChipChat application offers several strengths that contribute to secure and reliable communication. One of its main strengths is the implementation of end-to-end encryption using RSA-OAEP and AES-GCM. This ensures that messages remain confidential during transmission and can only be accessed by the intended recipient.

Another strength of the system is the integration of multiple cybersecurity protection mechanisms. In addition to encrypted communication, the application includes SQL injection detection, cross-site scripting (XSS) detection, brute-force attack monitoring and replay attack detection. These features provide additional layers of protection against common cyber threats and improve the overall security of the platform.

The system also supports real-time communication through Flask-SocketIO, allowing users to exchange messages, images, audio recordings and videos efficiently. Furthermore, the administrative dashboard provides security monitoring and attack logging capabilities, enabling administrators to review security-related events and system activities.

Finally, the web-based architecture allows the system to be accessed through standard web browsers without requiring dedicated software installation. This improves accessibility and usability for users while maintaining secure communication.

5.5 Limitation of Proposed System

Despite its strengths, the ChipChat application has several limitations. Firstly, the system was developed as a prototype for educational and research purposes and therefore does not provide some advanced features commonly found in commercial messaging applications, such as group chat, video calling, audio calling and multi-device synchronization.

Secondly, the system relies on locally stored cryptographic keys within the user's browser environment. If browser data is removed or the device is changed, users may lose access to previously stored private keys and may need to register again. A more advanced key management mechanism would be required for large-scale deployment.

In addition, although the system includes a password recovery function using One-Time Password (OTP) verification, OTP messages are currently not delivered directly to user's mobile phone numbers due to third-party SMS service limitations. The OTP process is therefore demonstrated within the system environment.

The system also limits media uploads to a maximum size of 15 MB for images, audio and video files. While this helps maintain system performance and efficient message transmission, it may restrict the sharing of larger media files.

Overall, the system successfully demonstrates secure communication and cybersecurity concepts, although several enhancements would be beneficial for future development.

Chapter 6 Conclusion

6.1 Summary of the Study / Project

This project focused on the design and development of ChipChat, a secure web-based chat application that implements end-to-end encryption to protect user communication. The system was developed using HTML, CSS, JavaScript, Bootstrap, Flask, Flask-SocketIO and SQLite.

To enhance communication security, RSA-OAEP was implemented for secure key exchange and AES-GCM was used for message encryption. In addition to encrypted communication, the system integrates several cybersecurity protection mechanisms including SQL injection detection, cross-site scripting (XSS) detection, brute-force attack monitoring and replay attack detection.

The completed system supports user registration, login, password recovery, real-time messaging, media sharing, user blocking and administrative security monitoring. Overall, the project successfully showed how encryption and cybersecurity mechanisms can be applied in a real chat application.

6.2 Achievement of Objectives

The objectives established at the beginning of the project were successfully achieved.

The first objective was to develop a secure chat application using RSA-OAEP and AES-GCM cryptographic algorithms with end-to-end encryption. This objective was achieved through the successful implementation of encrypted messaging and secure key exchange mechanisms within the ChipChat application.

The second objective was to ensure message confidentiality, integrity and authenticity through secure encryption technologies. This objective was accomplished through the implementation of end-to-end encryption, password hashing and secure authentication mechanisms.

The third objective was to evaluate the system based on encryption performance, message delivery efficiency and usability. Testing results demonstrated that the application was capable of supporting secure real-time communication while maintaining acceptable performance and user experience.

Therefore, all project objectives were successfully achieved.

6.3 Recommendations for Future Work

Several improvements can be considered for future development of the ChipChat application. Firstly, group chat functionality could be introduced to support communication among multiple users simultaneously.

Secondly, integration with a commercial SMS service provider could allow OTP verification codes to be delivered directly to user's mobile phone numbers, improving the password recovery process.

Future versions of the system could also support larger media file uploads and improved file management features. In addition, more advanced key management mechanisms and multi-device synchronization could be implemented to improve usability and communication security.

Finally, the system could be migrated to a more scalable database platform and cloud infrastructure to support larger numbers of users and higher communication volumes.

6.4 Final Remarks

In conclusion, the ChipChat project successfully demonstrated the development of a secure web-based communication platform that combines cryptography and cybersecurity principles. Through the implementation of end-to-end encryption and security monitoring mechanisms, the system provides secure and reliable communication while protecting user privacy.

The project also shows that secure communication can be achieved by combining encryption technologies with practical cybersecurity controls. Overall, the project achieved its intended objectives and provides a strong foundation for future enhancements and research in secure communication systems.

References

- | |
|---|
| <p>Perkasa, M.J.P., Ramdan, H.M., Maliki, A.J. and Somantri, 2025. Implementation of Secure End-to-End Encrypted Chat Application Using Diffie–Hellman Key Exchange and AES-256 in a Microservice Architecture. <i>Engineering Proceedings</i>, 107(1), p. 98. Available at: https://www.mdpi.com/2673-4591/107/1/98.</p> |
| <p>Carranza, H., Bustamante, M., Carranza, A. and Muniganti, S., 2024. Secure Chat Application with an End-to-End Encryption. In: <i>Proceedings of the 10th World Congress on Electrical Engineering and Computer Systems and Sciences (EECSS 2024)</i>. Paper No. CIST_140. Available at: https://www.avestia.com/EECSS2024_Proceedings/files/paper/CIST/CIST_140.pdf.</p> |
| <p>Ali, A. and Sagheer, A. (2017). Design of Secure Chatting Application with End to End Encryption for Android Platform. <i>Iraqi Journal for Computers and Informatics</i>, 43(1), pp.22–27. Available at: https://www.researchgate.net/publication/340444937_Design_of_Secure_Chatting_Application_with_End-to-End_Encryption_for_Android_Platform.</p> |
| <p>Bais, A.S., Sao, A., Jain, H., Sahu, S. and Bajpai, P., 2024. SECURE CHAT USING ENCRYPTION. <i>International Journal of Research Publication and Review</i>, 6(4), pp. 1604–1608. Available at: https://ijrpr.com/uploads/V6ISSUE4/IJRPR43982.pdf.</p> |
| <p>Yusfrizal, Y., Meizar, A., Kurniawan, H. and Agustin, F., 2018. Key Management Using Combination of Diffie–Hellman Key Exchange with AES Encryption. In: <i>Proceedings of the 2018 6th International Conference on Cyber and IT Service Management (CITSM)</i>. IEEE, pp. 1-6. Available at: https://ieeexplore.ieee.org/document/8674278/.</p> |
| <p>Harshvardhan, B., Saideep, C., Atharv, D., Ankit, J. and Bhise, A., 2023. A Secure Messaging Application with Unbreakable End to End Encryption. <i>International Journal for Research in Applied Science & Engineering Technology (IJRASET)</i>, 11(5), pp. 234-239. Available at: https://www.ijraset.com/best-journal/a-secure-messaging-application-with-unbreakable-end-to-end-encryption.</p> |
| <p>Pancoro, W., 2024. Securing Text Messages Using E2EE (End-To-End Encryption) on Instant Messaging Applications. <i>International Journal of Information Technology and Business</i>, 6(2), pp. 17-23. Available at: https://ejournal.uksw.edu/ijiteb/article/view/2670.</p> |
| <p>Deepika, S., Varma, R.Y., Rajeswari, V.D.L., Ramya, S. and Sravya, M.Y.V., 2021. SECURE DATA TRANSMISSION USING HYBRID CRYPTOGRAPHY. <i>Journal of Emerging Technologies and Innovative Research (JETIR)</i>, 8(8), pp. a868–a875. Available at: https://www.jetir.org/papers/JETIR2108114.pdf.</p> |
| <p>Mokhtarnameh, R., Muthuvelu, N., Ho, S.B. and Chai, I. (2010). A Comparison Study on Key Exchange-Authentication protocol. <i>International Journal of Computer Applications</i>, 7(5), pp.5–11. Available at: https://www.ijcaonline.org/volume7/number5/pxc3871459.pdf</p> |
| <p>Rahegaonkar, A., 2022. <i>A study on the complexity of Cryptographic Algorithms used for Secure Messaging</i>. MSc dissertation, National College of Ireland. Available at:</p> |

<https://norma.ncirl.ie/6044/1/ameetrahegaonkar.pdf>.

Puyosa, I (2023). Protecting point-to-point messaging apps: Understanding Telegram, WeChat, and WhatsApp in the United States. [online] Atlantic Council. Available at: <https://www.atlanticcouncil.org/in-depth-research-reports/report/point-to-point-messaging-apps/>.

LY Corporation (2026). LINE Help Center. [online] Line.me. Available at: <https://help.line.me/line/smartphone?contentId=50000087&contentid=20230304&lang=en>.

Trend Micro. (2025). Top 15 Most Common Types of Cyberattacks. [online] Available at: https://www.trendmicro.com/en_gb/what-is/cyber-attack/types-of-cyber-attacks.html.

Fortinet (2023). Top 20 Most Common Types Of Cyber Attacks. [online] Fortinet. Available at: <https://www.fortinet.com/resources/cyberglossary/types-of-cyber-attacks>.

Gantt Chart

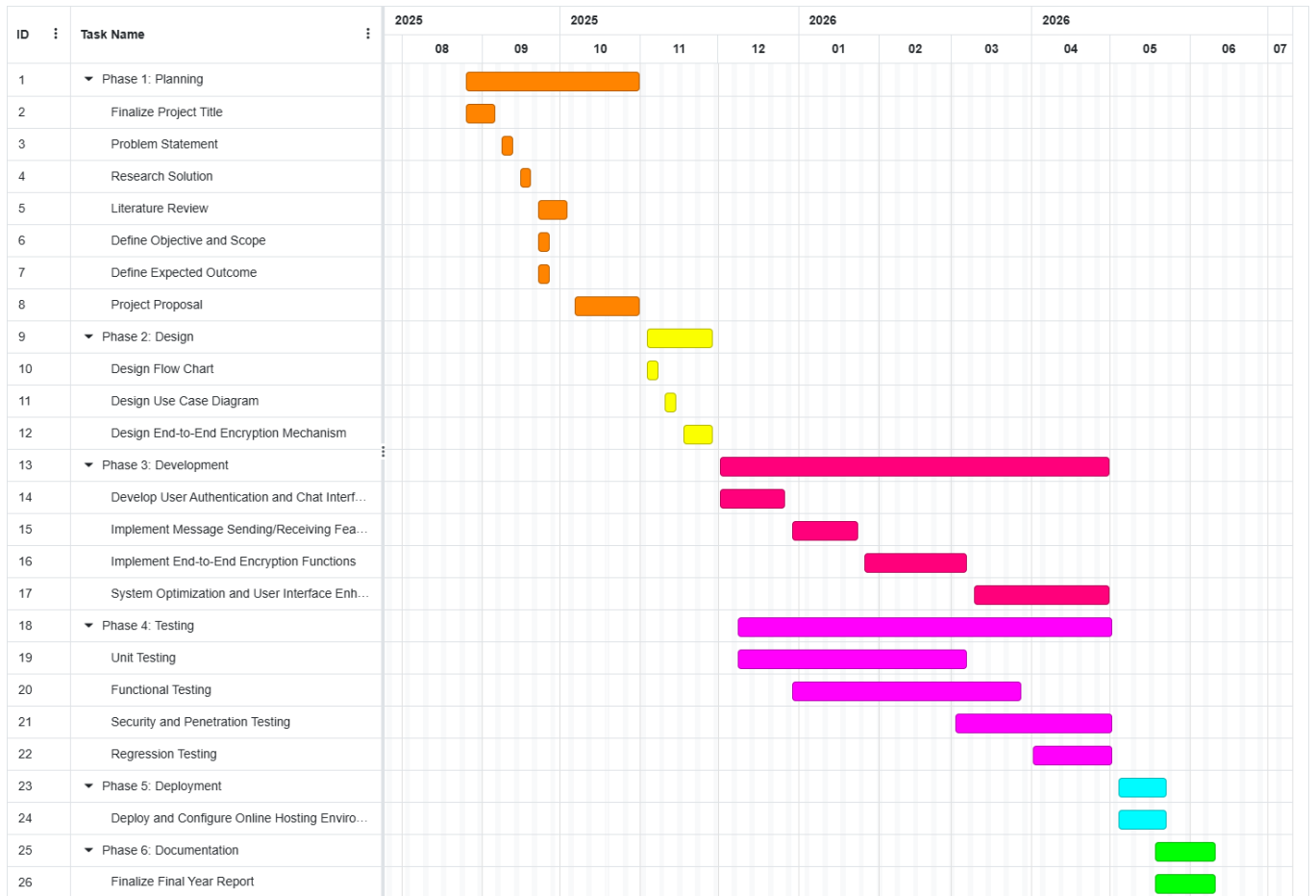


Figure 35 Gantt Chart

Appendix

PROPOSAL

[PROPOSAL REPORT](#)

TEST CASE

[TEST CASE](#)

CHAPTER 4 FUNCTION EXPLANATION

[FUNCTION EXPLANATION](#)

POSTER

[PROJECT POSTER](#)

PRESENTATION SLIDE

[PROJECT PRESENTATION SLIDE](#)

VIDEO DEMONSTRATION

[HTTPS://YOUTU.BE/2U_NH3NH6II](https://youtu.be/2U_NH3NH6II)

GITHUB

<https://github.com/jiajian4680/Secure-Chat-Application.git>